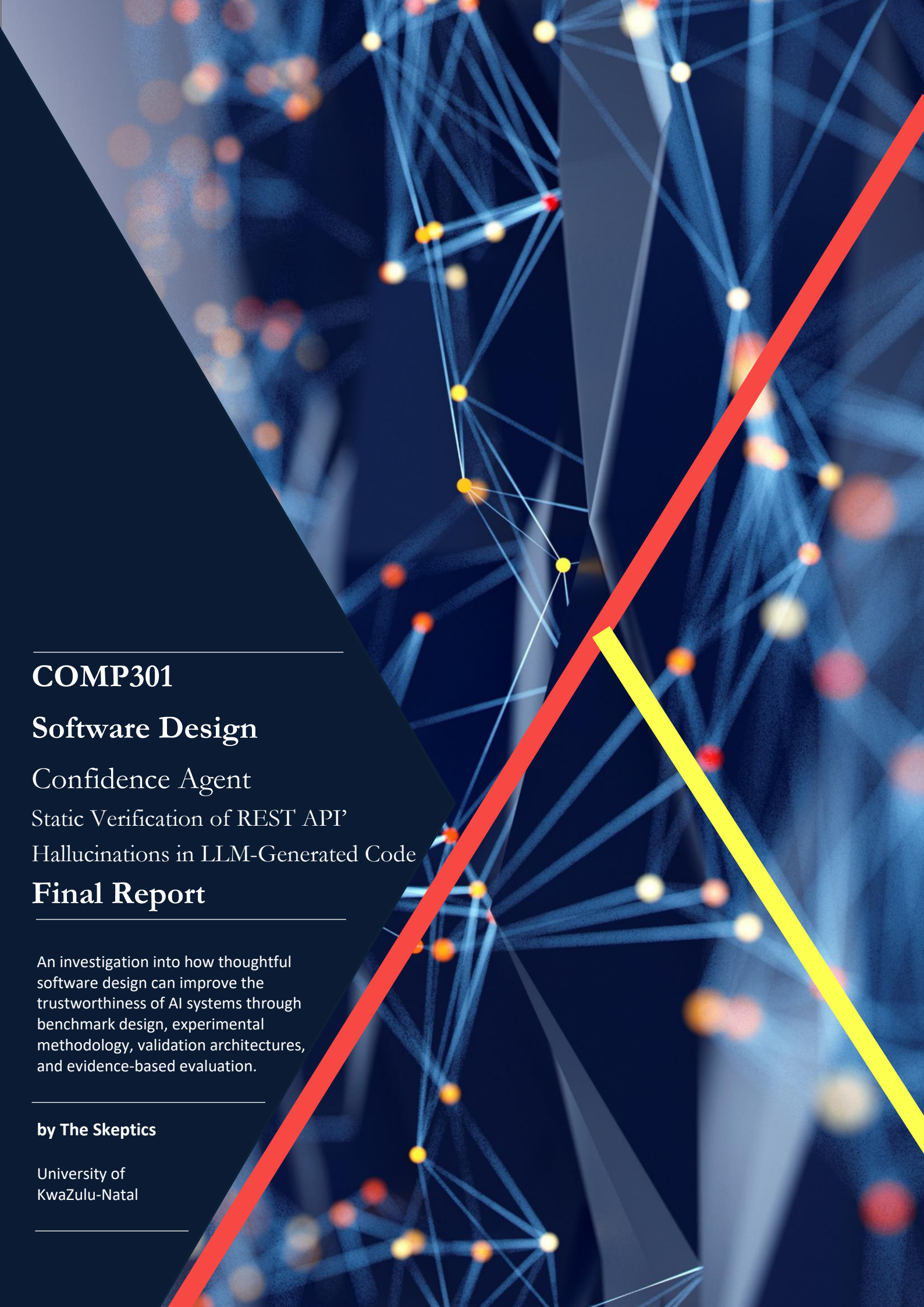


COMP301

Software Design

Confidence Agent

Static Verification of REST API' Hallucinations in LLM-Generated Code

Final Report

An investigation into how thoughtful software design can improve the trustworthiness of AI systems through benchmark design, experimental methodology, validation architectures, and evidence-based evaluation.

by The Skeptics

University of KwaZulu-Natal

Project Formalities

Domain: Static Verification of API Hallucinations in LLM-Generated Code

Group Name: The Skeptics

Group Members:

Member	Student number
Jawad Kachbal	223037045
Joshua Blom	224039813
Mahomed Valli Mahomed	224005878
Andiswa Hlengwa	221002430
Abiel Govender	224058851

Abstract

Large language models write plausible code that calls web APIs which do not exist, at versions that were retired, with parameters the provider never documented. This report investigates whether a **Confident Agent** can stop such REST API hallucinations from reaching the user. The agent pairs a local model (Qwen3.5-9B served by LM Studio) with a prompt and a tool that verifies every API claim in a candidate program against **frozen, hashed OpenAPI specifications** across six layers, scores the evidence with an **Evidence-Weighted Confidence Score (EWCS)**, and applies a decision policy that delivers, flags, corrects once, or withholds the code.

We built a frozen benchmark of 1,000 Python tasks over six public REST APIs and answered each of the 990 scored tasks twice, once with the plain model and once with the Confident Agent. The experiment was run under three quantizations of the same model (Q4_K_M, Q6_K and Q8_0): 2,970 task pairs that repeat the same 990 tasks, not three independent replications. An evaluator that shares no code with the agent executed every candidate in a sandbox against the answer key. In each run the share of tasks whose delivered code contained an API hallucination fell from **50.8–55.2% to 1.6–2.3%** (exact McNemar $p < 10^{-129}$, Holm-adjusted), while correct code reaching the user rose from **24.9–36.9% to 50.4–51.5%**. These figures measure the combined intervention of prompt, verification, one correction and abstention, not the contribution of each part; the agent's first candidates, written under its prompt but before verification, still hallucinated in 46.7–48.9% of tasks, against 50.8–55.2% for the baseline.

Release is governed by hard verification constraints, not by the confidence score: re-scoring every decision under the five planned weighting schemes changes only whether delivered code carries a flag, never what is withheld. Safety has costs: abstention on 38–40% of tasks, including almost every REST Countries task, and about **six times the time and energy** (37–48 s and R10.51–R14.87 of electricity per 1,000 prompts at the Durban Central tariff, against 6–7 s and R1.95–R2.42). Verification establishes conformance to the API contract, not that the code does what the user asked: only 19–23% of the agent's delivered programs met all three task-behaviour checks.

Keywords: LLM hallucination, REST API, OpenAPI, static verification, confidence estimation, abstention, McNemar test, energy cost.

Table of Contents

1 Introduction 1
1.1 Motivation 1
1.2 Contributions 1
2 Research Question and Hypotheses 1
3 Application Domain and Definitions 2
3.1 Hallucination and API hallucination 2
3.2 Scope, and why external evidence is needed 3
4 Benchmark Design and Justification 3
4.1 Requirements and APIs 3
4.2 Tasks and the answer key 4
5 Baseline Evaluation 4
6 Confident Agent Architecture and Design 5
6.1 Architecture overview 5
6.2 Claims, layers and evidence 6
6.3 The Evidence-Weighted Confidence Score 6
6.4 The decision policy: score, constraints and gate 6
7 Experimental Methodology 7
7.1 Design, and the structure of the observations 7
7.2 Procedure and measurements 8
7.3 Independent evaluation and manual review 8
7.4 Pre-registration and statistical analysis 8
8 Experimental Results and Comparative Analysis 9
8.1 Headline results 9
8.2 What the results show about the components 10
8.3 Where the architecture does not succeed 10
8.4 The confidence score, the gate and the policy 11
8.5 Correction, protocol and cost 12
9 Statistical Analysis 12
9.1 Primary and secondary hypotheses 12
9.2 What the significance tests do and do not show 12
9.3 Robustness checks 13
10 Threats to Validity 13

10.1 Construct validity 13
10.2 Internal validity 13
10.3 External validity 14
10.4 Conclusion validity and reproducibility 14
11 Conclusion 14
AI Disclosure 16
References 17
Appendix A: Summary of Results 19
Appendix B: Benchmark and Methodology Details 20
B.1 Mapping to the taxonomy of Progress Report Two 20
B.2 Evidence and its provenance 20
B.3 Task construction and composition 21
B.4 Model and run configuration 22
B.5 Experimental pipeline 23
B.6 Procedure and measurements 23
B.7 The independent evaluator 24
B.8 Manual review of undetermined candidates 24
B.9 Changes to the evaluation plan 24
B.10 Statistical methods 25
B.11 Summary of threats to validity 25
Appendix C: Architecture Details 27
C.1 The Trust-by-Design patterns 27
C.2 Where the tool attaches, and why 27
C.3 Claim extraction 28
C.4 The six layers in detail 28
C.5 Frozen evidence, selected by host 29
C.6 The EWCS: signed score, weights, tiers and ceilings 29
C.7 Static and dynamic structure 31
C.8 Design evolution and verification of the verifier 32
Appendix D: Supplementary Results 34
D.1 Scorecard 34
D.2 Results by subgroup 35
D.3 Kinds of hallucination 35
D.4 The verifier as a hallucination detector 36

D.5 Discrimination and calibration of the EWCS 37
D.6 Sensitivity to the weights, thresholds and gate 37
D.7 Correction and agent behaviour 38
D.8 Behaviour beyond API correctness 39
D.9 The cost of the intervention 39
D.10 Quantization, run-to-run reliability and agreement between runs 42
D.11 Secondary outcomes and effect sizes 43
D.12 Cost statistics and reviewer agreement 44
Appendix E: Changes since Progress Report Two 45
Appendix F: Example Task and Prompts 47
Appendix G: Run Artefacts and Provenance 48
Appendix H: Error Kinds Used by the Evaluator 49
Appendix I: Manual Review Rules 50
Appendix J: Reproducing the Results 51
Appendix K: Glossary 52

List of Figures

Figure 1**:** Which API host and version each baseline answer called (three runs pooled). The model reproduces versions from its training data and extrapolates versions it has never seen. 5
Figure 2**:** Deployment and component view (UML). All components run on one workstation; the evaluator is independent of the toolbox. 5
Figure 3**:** Activity diagram (UML) of the decision policy. Hard rules precede the score; the renderer releases code only when its hash matches the verified candidate. 7
Figure 4**:** Each task's outcome under the baseline (left) and under the Confident Agent (right), three runs pooled; ribbon width is the number of task-runs. 9
Figure 5**:** Summary of results: supplementary charts (a)–(l). 19
Figure 6**:** Experimental pipeline. The answer key reaches only the evaluator; the model and the agent see the prompt alone. 23
Figure 7**:** The independent evaluator: sandboxed execution, three response scenarios and comparison with the answer key. 24
Figure 8**:** How the six layers read one candidate: each claim is checked by one layer against one kind of evidence. 28
Figure 9**:** Class diagram (UML) of the verification core in toolbox 3.0.0. 31
Figure 10**:** Sequence diagram (UML) of one intervention task, including the single permitted correction and the reply request at the turn limit. 32
Figure 11**:** Scorecard: baseline versus Confident Agent for each quantization (990 tasks each). Electricity priced at R4.1109/kWh including VAT. 34
Figure 12**:** Rates per subgroup and run: hallucinated code reaching the user, correct code reaching the user, and code withheld by the verifier (B = baseline, CA = Confident Agent). 35
Figure 13**:** Kinds of API hallucination in the code reaching the user (three runs pooled; a task can have several kinds). 35
Figure 14**:** The verifier as a detector of hallucinated final candidates: quality measures, confusion matrices, and the processing time spent on its errors. 36
Figure 15**:** EWCS of the final candidate: ROC curve for correct versus class A, and score distributions with the decision thresholds (three runs pooled). 37
Figure 16**:** Decisions on the final candidate under the five planned schemes and with the uncertainty gate removed (three runs pooled). 38
Figure 17**:** Cost and outcome by the number of correction rounds the agent made (three runs pooled). 38
Figure 18**:** Deviations from the Confident Agent prompt recorded by the benchmark, per run. 39
Figure 19**:** Time, electricity (Rand per 1,000 prompts) and tokens per prompt, with 95% bootstrap intervals 40
Figure 20**:** What one correct answer, one prevented hallucination and one extra correct answer cost in electricity and time. 41
Figure 21**:** Where the Confident Agent's time and electricity go, by outcome. 42
Figure 22**:** Paired differences for the primary and secondary outcomes, with 95% bootstrap intervals over tasks (thick) and operation families (thin). 44
Figure 23**:** Project timeline from the frozen dataset to the final analysis. The shaded band marks the three final benchmark runs on which the findings rest. 45

List of Tables

Table 1: Classes used to score the code that reaches the user (evaluation plan §2). 2
Table 2: APIs in the benchmark and the provenance of their evidence (retrieved 21 September 2026). 3
Table 3: Baseline: class of the code delivered to the user (990 tasks per run, pilot excluded). 4
Table 4: Headline outcomes per run (990 tasks each; baseline → agent). 9
Table 5: Class A code at three stages: the baseline answer, the agent's first candidate (before verification feedback) and the code the agent delivered. 10
Table 6: Sensitivity analysis: outcomes on the final verified candidate (three runs pooled, 2,970 task-runs). 11
Table 7: Primary test: class A code reaching the user, baseline versus Confident Agent (990 paired tasks per run). 12
Table 8: Mapping of the Progress Report Two taxonomy to the evaluator's error kinds. 20
Table 9: Composition of the 990 scored tasks (the 10 pilot tasks are excluded from the analysis). 21
Table 10: The pinned model and run configuration (identical for baseline and intervention; read from the run manifests). 22
Table 11: What was measured for every task and condition. 23
Table 12: Changes to the evaluation plan after it was fixed on 27 September 2026. 24
Table 13: Statistical methods (evaluation plan §6). 25
Table 14: Summary of the main threats to validity and their mitigation. 25
Table 15: How the Confident Agent realises the Trust-by-Design patterns (Vorster, 2026a). 27
Table 16: Options for attaching verification to the model. 27
Table 17: Severity ranking of the layers and the rank-sum weights (total 15). 29
Table 18: Evidence tiers and factors. 30
Table 19: Maximum attainable EWCS when every applicable layer verifies (L1 tier 2, L2 tier 4). 30
Table 20: Engineering checks of the verifier on known-valid and known-invalid code (toolbox reference regression). 33
Table 21: Share of decided final candidates (correct, A or B) that were correct, by EWCS band (three runs pooled). 37
Table 22: Class of the first verified candidate → class of the final candidate, for tasks in which the code changed. 38
Table 23: Task-behaviour checks on code that reached the user (share of programs that make a request). 39
Table 24: Cost of the intervention in time, energy and Rand (990 tasks per run; electricity at R4.1109/kWh incl. VAT). 40
Table 25: Repeat runs with identical settings: first series (superseded) versus final series. 42
Table 26: Agreement between the three final runs, task by task (990 tasks; Cohen's κ in brackets). 43
Table 27: Secondary outcomes (exact McNemar tests; intervals over tasks). 43
Table 28: Design decisions in Progress Report Two and in the final system. 45
Table 29: Identifiers of the frozen inputs (SHA-256 prefixes; full hashes in the run manifests and the dataset). 48
Table 30: Kinds of error recorded in the code reaching the user, read from the evaluator's findings with fixed rules. 49
Table 31: Glossary. 52

1 Introduction

1.1 Motivation

Developers now ask language models for integration code as routinely as they search documentation, and a large share of the answers call an API that is wrong in a way the developer cannot see: a renamed endpoint, a retired host, a parameter that never existed. The COMP301 brief frames the remedy as a design problem: *a Confident Agent is not one that is always correct; it is one that knows when it should not be trusted* (University of KwaZulu-Natal, 2026). We treat that statement as a testable claim. We hold the model fixed, add an explicit verification architecture around it, and measure with an independent judge whether fewer hallucinated API calls reach the user, how many correct answers are lost, and what the checking costs.

We chose software APIs, after abandoning an education setting in which web access made the intended traps trivial (The Skeptics, 2026a), because they offer an **authoritative, machine-readable specification** against which a claim can be checked without trusting the model (Vorster, 2026c). Progress Report Two fixed the design (The Skeptics, 2026b); Appendix E lists where this report supersedes it.

1.2 Contributions

- **A frozen benchmark** of 1,000 Python tasks over six public REST APIs, with an executable answer key (Section 4).
- **The Confident Agent toolbox**: six-layer verification against frozen evidence, a confidence score and a decision policy with hard constraints (Section 6).
- **A paired experiment** on 990 tasks under three quantizations, judged by an independent sandboxed evaluator, with a pre-registered analysis plan and a cost analysis in Rand (Sections 7–9).
- **An architectural finding**: release decisions can be governed by explicit verification constraints rather than by the model's judgement or a confidence threshold (Sections 6.4 and 8.4).

Supporting details and analyses are in the appendices; Appendix A summarises the supplementary charts on one A3 page.

2 Research Question and Hypotheses

> **Research question.** Can an LLM-based code-generation agent that verifies the REST API claims in its own code against frozen, authoritative specifications reduce the unacceptable API hallucinations that reach the user, compared with the same model without verification, while abstaining when verification is inconclusive, and at what cost?

Reaching the user places the harm at delivery: a hallucinated candidate that the agent withholds is not a failure, but one that it delivers is. *Frozen, authoritative specifications* distinguishes the approach from self-verification by the same model, which can rationalise its own errors (Vorster, 2026a), and *abstaining* makes refusal a measured outcome rather than a hidden one.

> **H1 (primary).** Agent-based validation equipped with explicit, evidence-based confidence estimation significantly reduces the proportion of tasks in which code containing an API hallucination (class A, Section 3.1) reaches the user, compared with the equivalent baseline model. Test: exact two-sided McNemar test on paired tasks within each quantization run, $\alpha = 0.05$, Holm-adjusted across the three runs.

- **H2 (correctness is preserved).** The proportion of tasks in which correct code reaches the user does not decrease. A verifier that refused everything would fail H2.
- **H3 (the confidence metric discriminates).** Among final candidates, correct code receives a higher EWCS than hallucinated code (ROC area above 0.5). We do not claim the EWCS is a calibrated probability.
- **H4 (the cost is measurable).** Time, tokens and energy per prompt increase; we estimate the ratios and their price rather than test their direction.

We report **incorrect acceptance** (hallucinated code is delivered) and **incorrect rejection** (correct code is withheld) separately, and a reduction in hallucinations bought only through blanket refusal would count as a negative result.

3 Application Domain and Definitions

3.1 Hallucination and API hallucination

A hallucination is generated content that is fluent and plausible but unfaithful to, or unsupported by, the source or the world (Ji et al., 2023; Huang et al., 2025); for code it includes invented packages (Spracklen et al., 2025), non-existent functions and wrong API usage (Liu et al., 2024; Patil et al., 2023). Our working definition is: **an LLM hallucination is content that appears plausible and confident but is incorrect, fabricated, or unsupported by verified ground truth.**

A claim about a **REST API** (Fielding, 2000) in code has six parts, which map onto our verification layers: the library imported, the identifier called, the access point (host, path and method), the input contract, the output contract, and the required context such as authentication. An **OpenAPI** document declares these parts in machine-readable form (OpenAPI Initiative, 2021), so a claim such as `GET https://api.github.com/orgs/{org}` can be checked deterministically without running the code. An **API hallucination** is a claim verified to be incorrect against such an authoritative source. *Cannot be verified* does not mean *false*: of the four states a claim can have (verified correct, verified incorrect, unverifiable, not applicable), only the second is a hallucination, and the third is a reason to abstain. The independent evaluator classifies delivered code as in Table 1 (taxonomy mapping in Appendix B.1).

Table 1: Classes used to score the code that reaches the user (evaluation plan §2).

Class	Meaning and examples	Counts as
A API hallucination	Breaks the API contract: wrong host, version or base path; undeclared path, method, parameter or field; missing required part or credential; wrong value type; undocumented response field.	Primary outcome
B task-fidelity error	A valid API call that is not the one the task asked for (another endpoint, method, value or body).	Secondary outcome
Correct (incl. **C** equivalent)	The expected request, allowing a closed list of equivalences (e.g. `token` X for `Bearer X`).	Correct

Class	Meaning and examples	Counts as
Runtime failure	The code makes no request: syntax error, crash or no call.	Error
Undetermined	A name a partial specification does not list; resolved by manual review.	Resolved

3.2 Scope, and why external evidence is needed

We study **Python with `requests`** and **REST APIs with OpenAPI contracts**. Verification is **static**: the agent never executes generated code, because model output may be destructive, a verdict would depend on a live service on a particular day, and execution would need real credentials, which the benchmark must not contain (Vorster, 2026c). A model's knowledge is a snapshot that ends at a cut-off and over-represents what was common on the web (Cheng et al., 2024), while public APIs keep changing. The model therefore tends to write the API that was popular in its training data, with no internal signal that it is stale (Section 5). Only evidence from outside the model can detect this, which is why the agent is anchored to frozen, current specifications.

4 Benchmark Design and Justification

4.1 Requirements and APIs

The brief asks for a representative, objective and reproducible benchmark; the lecturer added that ground truth must be frozen and hashed, credentials must be placeholders, and curated specifications must be labelled (Vorster, 2026c). The benchmark uses six public REST APIs spanning no authentication, API keys, Bearer tokens and OAuth2 (Table 2). Each task carries an exact expected request, so correctness is decided by comparing requests; every specification is referenced by its SHA-256 hash; no task contains a real credential; and the dataset is generated by a deterministic script, not by a language model (Appendix B.2).

Table 2: APIs in the benchmark and the provenance of their evidence (retrieved 21 September 2026).

API	Tasks	Families	Ground-truth source	Tier	Authentication
GitHub REST	250	65	Official OpenAPI, full document (819 paths)	1	Bearer token
Swagger Petstore	150	11	Official OpenAPI 3 document	1	API key / OAuth2
Open-Meteo	150	3	Official OpenAPI (forecast, air quality, marine)	1	None
JSONPlaceholder	150	36	Curated partial specification from the official guide	2	None
DummyJSON	150	23	Curated partial specification from the documentation	2	None
REST Countries v5	150	4	Curated partial specification from the v5 documentation	2	API key (Bearer)

Tier 1 = provider-published specification; tier 2 = researcher-curated partial specification, labelled as such. Total: 1,000 tasks in 142 operation families.

For three APIs no adequate official OpenAPI document exists, so we built small specifications from the cited documentation and labelled them as researcher-authored. A name such a specification does not list is not proof that it does not exist, so the evaluator marks it undetermined for manual review

(Section 7.3). REST Countries recently moved to an authenticated v5 interface on a new host; we kept it because it is exactly the case in which a model's training data is out of date.

4.2 Tasks and the answer key

The generator builds each task from a checked template for a documented operation; related variants share an **operation family** (141 among the scored tasks) so the analysis can allow for their correlation. Prompts name the API and describe the operation and its inputs, but never give the URL, method or parameter names, and never mention hallucination. Each asks for a function `solve(inputs)` and states how to handle an empty response and an error, which gives behaviour checks beyond API correctness (Appendix F). The 990 scored tasks are 494 reads, 232 creates, 197 updates, 37 searches and 30 deletes; their proportions differ from the targets of Progress Report Two, and we report subgroups rather than relabel tasks (Appendix B.3).

The task file is the answer key. Before any model output was scored, the evaluator's **self-check** executed all 1,000 reference solutions and classified every one correct. The planned two-reviewer human review of a stratified 100-task sample of the answer key was not completed (Section 10.1). Every task is answerable, so the benchmark measures the natural hallucination rate on ordinary requests but cannot reward a correct refusal. Ten pilot tasks used to develop the agent are excluded from every statistic.

5 Baseline Evaluation

The experiment uses one model served one way: Qwen3.5-9B (GGUF) in LM Studio, at temperature 0.2 without a fixed seed, with reasoning off and a 30,208-token context. Both conditions receive the identical prompt file; the baseline system prompt asks only for the code in one Python block. The experiment was run three times, with the Q4_K_M, Q6_K and Q8_0 quantizations of the same weights, between 30 September and 3 October 2026 (full configuration in Appendix B.4).

Table 3**:** Baseline: class of the code delivered to the user (990 tasks per run, pilot excluded).

Class	Q4_K_M	Q6_K	Q8_0
Correct	247 (24.9%)	365 (36.9%)	341 (34.4%)
A: API hallucination	513 (51.8%)	503 (50.8%)	546 (55.2%)
B: valid call, wrong task	79 (8.0%)	56 (5.7%)	58 (5.9%)
Runtime failure or no code	151 (15.3%)	66 (6.7%)	45 (4.5%)

Between 51% and 55% of baseline answers contained an API hallucination at every quantization (Table 3): the rate reflects what the model knows, not its numerical precision, while the lower quantization pays in ordinary programming errors instead. Most baseline hallucinations (1,303 of 1,562 across the three runs) are a wrong host, version or base path, and Figure 1 shows where they come from. These are errors the model cannot detect by introspection, and that a frozen specification detects deterministically.

- **Retired versions.** 78% of Petstore answers called the Swagger 2 host `petstore.swagger.io/v2`, which dominates tutorials; 63% of REST Countries answers called the deprecated `restcountries.com/v3.1`.

- **Extrapolated versions.** Told "REST Countries v5", the model invented `restcountries.com/v5.1/...` in 35% of answers; only 4 of 447 reached the real host.
- **Sibling confusion.** 45% of DummyJSON answers called JSONPlaceholder. Where the model's knowledge is current (JSONPlaceholder, GitHub), the host was almost always right.

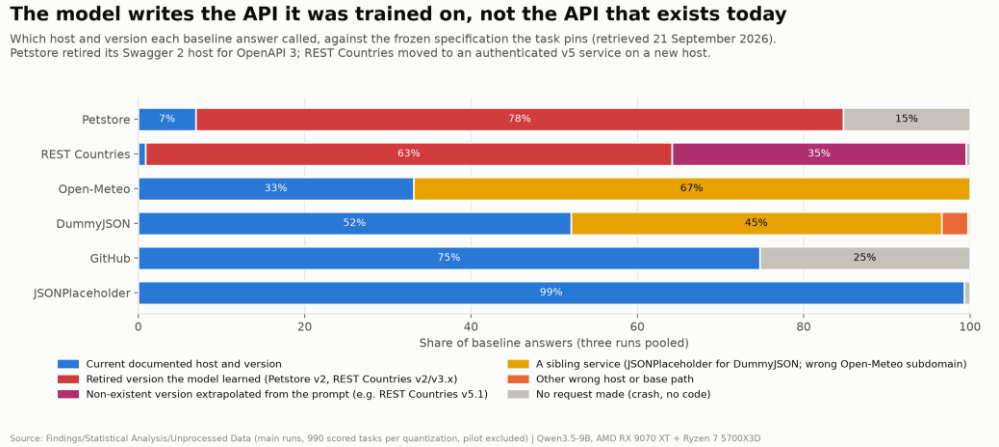


Figure 1: Which API host and version each baseline answer called (three runs pooled). The model reproduces versions from its training data and extrapolates versions it has never seen.

6 Confident Agent Architecture and Design

6.1 Architecture overview

The organising principle comes from the course's Trust-by-Design notes: generation alone must not confer authority on an output (Vorster, 2026a; pattern mapping in Appendix C.1). The model proposes code, a deterministic tool judges it against external evidence, and the tool, not the model, decides what may be released (Figure 2). The toolbox runs as an LM Studio tools provider or, in the experiment, through a Node.js bridge with the same `verify_candidate` implementation; the evaluator shares no code with it.

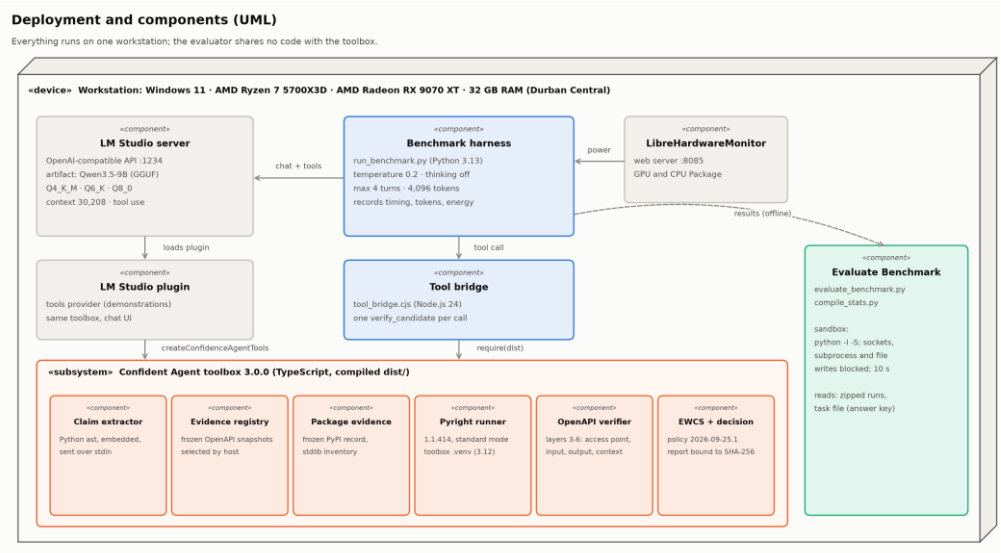


Figure 2: Deployment and component view (UML). All components run on one workstation; the evaluator is independent of the toolbox.

We attached verification as an **in-loop tool through function calling** rather than as a post-hoc controller, an MCP server or a multi-agent pipeline (Appendix C.2), because the package a developer would run is a model, a system prompt and one tool. The model then controls the conversation and may skip or repeat steps, so three safeguards keep the delivery decision with the tool: the benchmark scores the tool's decision on the last verified candidate, the tool's report embeds the candidate's SHA-256, and every protocol deviation is recorded.

6.2 Claims, layers and evidence

The extractor parses the candidate with Python's `ast` module (it never runs it); values that exist only at run time become typed holes, so `f"https://api.github.com/orgs/{org}"` is matched against OpenAPI path templates. Six layers then check the claims, each against one kind of evidence (weights in brackets):

- **L1 Interface (3):** imported modules exist. **L2 Identifier (3):** Pyright 1.1.414 finds no undefined name, attribute or call.
- **L3 Access point (4):** host, base path, path and method match a declared operation of the frozen specification.
- **L4 Input contract (2)**, **L5 Output contract (1)** and **L6 Contextual (2):** parameters and request body, response fields read, and security requirements match the operation.

Each layer returns one verdict (verified correct, verified incorrect, unverifiable or not applicable) with an evidence tier. The evidence is the same frozen, hashed snapshots as the dataset, **selected by the host the candidate itself calls**; the model never supplies a specification. A host without frozen evidence makes layers 3 to 6 unverifiable, so an invented host cannot be "verified". Dynamic specification discovery was rejected because a live specification can change between runs and four of the six APIs publish none (Appendix C.5).

6.3 The Evidence-Weighted Confidence Score

The EWCS is the proportion of the applicable verification weight supported by positive evidence. With A the applicable layers, w_l a layer's weight, t_l the factor of its evidence tier and c_l = 1 if it is verified correct (0 otherwise):

$$EWCS = \frac{\sum_{l \in A} w_l \cdot t_l \cdot c_l}{\sum_{l \in A} w_l}$$

The weights convert a severity ranking of the layers with the rank-sum rule (Stillwell, Seaver and Edwards, 1981; Barron and Barrett, 1996); a wrong access point ranks first because nothing downstream can then be right. Tier factors discount weaker evidence: 1.0 for a provider's specification, 0.8 for a curated specification or registry record, 0.5 for a clean Pyright run, 0 for none. The highest attainable scores are therefore known in advance: only fully modelled official specifications can reach 0.85, and curated ones cannot exceed 0.74 (Appendix C.6). The EWCS measures **evidence coverage, not the probability that the code is correct**.

6.4 The decision policy: score, constraints and gate

The policy (Figure 3) applies three ordered rules, and the first two do not consult the score:

1. **Safety constraint.** If any layer is verified incorrect, the code is not released: on a first candidate the agent may correct it once and must have the new code verified; on a second it abstains.
2. **Uncertainty gate.** If any applicable layer is unverifiable, failed to run or had incomplete extraction, the agent abstains.
3. **Thresholds.** Otherwise EWCS ≥ 0.85 delivers, 0.60 ≤ EWCS < 0.85 delivers with a flag, and a lower score abstains.

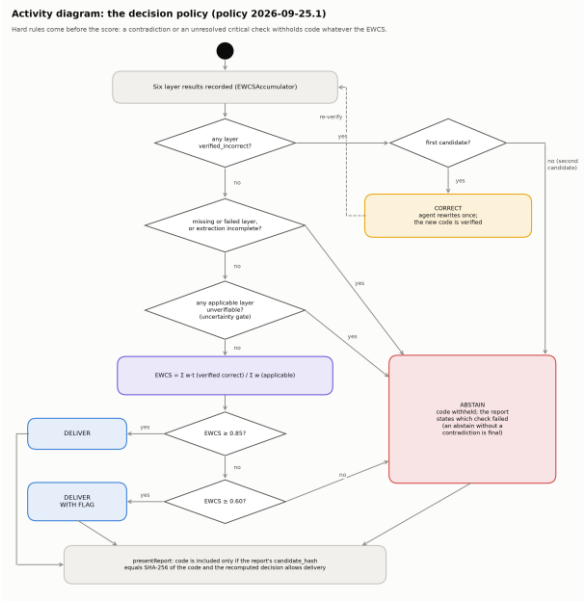


Figure 3: Activity diagram (UML) of the decision policy. Hard rules precede the score; the renderer releases code only when its hash matches the verified candidate.

The design thus has three parts that are easily conflated. The **confidence score** (EWCS) summarises how much evidence supports a candidate. The **hard verification constraints**, the safety constraint and the uncertainty gate, decide whether code may be released at all. The **decision policy** composes them in a fixed order, so the score is consulted only for candidates the constraints already allow. A candidate that passes the gate has every applicable layer verified and so scores at least 0.65, which means the 0.60 threshold can never bind: in this design the score decides only whether delivered code carries a flag (confirmed empirically in Section 8.4).

7 Experimental Methodology

7.1 Design, and the structure of the observations

The experiment is a **paired, within-task comparison**: each of the 990 scored tasks is answered by the same model with the same settings in the baseline and in the intervention condition, and the primary outcome is whether class A code reached the user (pipeline in Appendix B.5).

It was run three times, under the Q4_K_M, Q6_K and Q8_0 quantizations, which gives **2,970 task pairs**. These are **not three independent replications**: each run uses the same 990 tasks and prompts and the same model apart from numerical precision, so a task's outcomes are strongly correlated across runs. Whether the baseline hallucinated on a task agrees between quantizations with $\kappa = 0.69$–0.74, and 412 tasks hallucinated in all three runs while 363 hallucinated in none (Appendix D.10). Within a run

each task was sampled once per condition, and tasks are grouped into 141 operation families and six APIs. We therefore test each run separately on its 990 pairs, treat agreement across the runs as evidence of **robustness to quantization** rather than replication, use pooled figures only descriptively, and allow for the families with a cluster bootstrap.

The intervention is a **package**: the Confident Agent prompt (which prescribes the workflow and names the hosts with frozen evidence), the verification tool, one correction and abstention. Each condition generates its own code, because the prompt also shapes the first answer. The experiment therefore measures the **combined architecture** and was not designed to separate its components; Section 8.2 reports what the recorded data can and cannot say about them.

7.2 Procedure and measurements

In the **baseline**, the code block of the model's single reply is what the user receives. In the **intervention**, the model may call `verify_candidate`; the harness stores each candidate byte for byte with the tool's result. The prompt allows one correction, and only if a layer is verified incorrect; after four turns, a fixed message asks for the tool's report. What reaches the user is the code the tool delivered on the last verified candidate. For every task and condition we record the delivered code's class, the verifier's decision, EWCS and layer states, corrections and protocol deviations, three behaviour checks, time, tokens, and GPU and CPU energy priced at R4.1109 per kWh including VAT (eThekwini Municipality, 2026; Appendix B.6).

7.3 Independent evaluation and manual review

The grader must not trust the agent's own verdict (Vorster, 2026c), so the evaluator shares no code with the toolbox and uses a different method. It executes each candidate in an isolated process with the task's test inputs and a recording stand-in for `requests` that sends nothing, which yields the exact requests and every response field read. It runs each candidate against a success, an empty and an error response for the behaviour checks, and compares the requests with the expected request and the pinned specification (Appendix B.7). It scored all 1,000 reference solutions correct before scoring any model output.

Names that a curated specification does not list left **396 undetermined candidates**: 46 had reached the user and 350 had been withheld. Each open question was answered once from the provider's documentation or harmless live read requests and recorded as a rule (Appendix I). The first review was AI-assisted; a group member independently reviewed the 46 decisive candidates and agreed on 43 (Cohen's $\kappa = 0.88$), and the group settled the three disagreements as correct, which changes no hallucination figure.

7.4 Pre-registration and statistical analysis

Scoring rules, outcomes and tests were fixed in an evaluation plan on 27 September 2026, before benchmark run 2, and later changes were logged (Appendix B.9). The primary test is the exact two-sided McNemar test within each run (McNemar, 1947), $\alpha = 0.05$, with Holm's correction across the three runs (Holm, 1979). Differences carry 95% bootstrap intervals over tasks and over the 141 operation families (Efron and Tibshirani, 1993). Subgroups are descriptive; costs use the Wilcoxon signed-rank test (Wilcoxon, 1945), the EWCS the ROC area, and agreement Cohen's κ (Cohen, 1960; full plan in Appendix B.10).

8 Experimental Results and Comparative Analysis

All figures use the 990 scored tasks per run after the manual review. Pooled totals repeat the same tasks and are descriptive (Section 7.1); supporting analyses are in Appendix D.

8.1 Headline results

Table 4: Headline outcomes per run (990 tasks each; baseline → agent).

Outcome	Q4_K_M	Q6_K	Q8_0
Hallucination (A) reaching the user	51.8% → 1.6%	50.8% → 2.3%	55.2% → 2.0%
Tasks repaired / regressed (A)	505 / 8	490 / 10	529 / 3
Correct code reaching the user	24.9% → 50.9%	36.9% → 50.4%	34.4% → 51.5%
Abstention (nothing shown)	37.6%	39.5%	38.7%
Incorrect acceptance / incorrect rejection	16 / 21	23 / 20	20 / 28
Mean time per prompt	5.8 → 36.9 s	6.2 → 38.3 s	7.4 → 48.4 s
Electricity per 1,000 prompts (incl. VAT)	R1.95 → R10.51	R2.20 → R13.78	R2.42 → R14.87

In every run, hallucinated code reaching the user fell by 48.5–53.1 percentage points and correct code reaching the user rose by 13.5–26.0 points. The price is that 38–40% of tasks end with no code at all, and each prompt takes about six times as long.

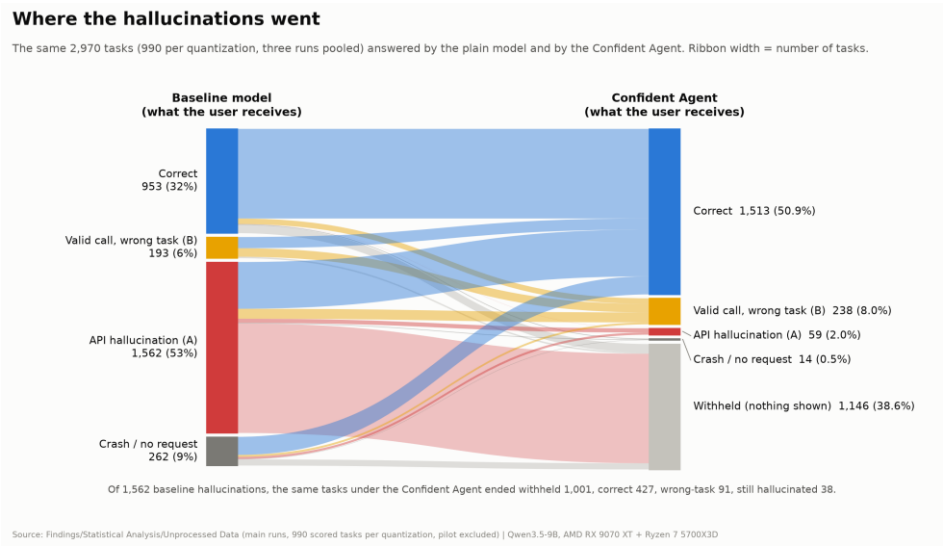


Figure 4: Each task's outcome under the baseline (left) and under the Confident Agent (right), three runs pooled; ribbon width is the number of task-runs.

Of the 1,562 baseline hallucinations across the runs (Figure 4), the agent withheld the code in 1,001, delivered correct code in 427, a valid call that did not match the task in 91, and still a hallucination in 38. Of the 953 correct baseline answers, it delivered correct code in 819 and withheld 76. About two-thirds of the prevented hallucinations were thus prevented by **withholding** code and one-third by **correcting** it. Withholding is the intended behaviour when evidence contradicts a candidate, but it is not an answer, so we count the 1,146 withheld tasks as a cost.

8.2 What the results show about the components

Because the intervention is a package (Section 7.1), the headline shows what the combined architecture achieves. Table 5 locates where the reduction happens, using the one intermediate observation the benchmark recorded: the **first candidate** the model submitted for verification. It was written under the Confident Agent prompt, including the host list, but before any verification feedback, and is scored by the same evaluator.

Table 5**:** Class A code at three stages: the baseline answer, the agent's first candidate (before verification feedback) and the code the agent delivered.

Stage	Q4_K_M	Q6_K	Q8_0
Baseline answer: class A	51.8%	50.8%	55.2%
Agent's first candidate: class A	48.9%	46.7%	47.9%
… counting undetermined as A	52.8%	54.5%	56.1%
Agent's delivered code: class A	1.6%	2.3%	2.0%
Wrong host, version or base path (tasks)	449 → 256 → 1	408 → 178 → 0	446 → 167 → 0
Other kinds of hallucination (tasks)	58 → 209 → 15	87 → 182 → 23	93 → 203 → 20

The last two rows count tasks at the three stages: baseline → first candidate → delivered; a task can have both kinds. Undetermined first candidates (names a curated specification does not list) were not manually reviewed. Computed by `Findings/EvaluateBenchmark/stage_analysis.py`.

The prompt on its own barely changed *how often* the model hallucinated: 46.7–48.9% of first candidates were class A (52.8–56.1% counting unresolved cases), against 50.8–55.2% of baseline answers. It changed *which* hallucination the model made: told the current hosts, it wrote a wrong host far less often (167–256 tasks per run, against 408–449) but invented paths and parameters on the right host instead, so other hallucinations rose from 58–93 to 182–209 tasks per run. The fall to 1.6–2.3% happens between the first candidate and the delivered code, where verification, correction and withholding act; there, hallucinations other than a wrong host fell to 15–23 per run, 74–79% below the baseline.

This is a **decomposition by stage, not an ablation**. The first candidate was written by a model told that its code would be verified, the correction step uses the tool's evidence, and we ran no condition with the tool but the baseline prompt or with the prompt but no tool. The separate contributions of prompting, verification, correction and abstention therefore remain unmeasured. The data support a narrower claim: the combined architecture works, and the reduction coincides with verification and release rather than with the model's first answer.

8.3 Where the architecture does not succeed

- **REST Countries: safe, but no useful answer.** The baseline hallucinated on 99–100% of REST Countries tasks and the agent delivered nothing for 99–100%. Even with the current host named in its prompt, the model invented v5 paths (all 258 withheld candidates reviewed used non-existent paths), and the verifier correctly refused them, at a cost of 7.1 hours of computation. The architecture prevented unsupported code from being released, but it cannot supply code the model does not know how to write: for this part of the domain it provides safety without usefulness.

- **Residual hallucinations.** 57 of the 59 hallucinations still delivered are Petstore tasks, mostly request bodies that put a run-time value where the schema expects a string. The verifier deliberately treats run-time values as opaque, so it cannot see which value lands in a field (Appendix D.3).
- **Conformance to the API is not the user's task.** The verifier checks the API contract, not the user's intent, so a valid call to the wrong endpoint passes by design: task-fidelity (class B) errors in delivered code rose from 193 to 238 across the runs, and counting them lowers the verifier's recall from 94–96% to 76–80% (Appendix D.4). Only 19–23% of the agent's delivered programs also met all three behaviour checks, mostly because few returned `None` on an empty response. Requiring the right request *and* all three behaviours, the agent delivered fully correct code for only 10.3–11.3% of tasks (baseline 2.1–6.5%; Appendix D.8).

A clean verification report is therefore evidence about API claims only, as the toolbox's report tells the user; "correct code" in this report means the right request, not a program that does everything the task asked.

8.4 The confidence score, the gate and the policy

H3 holds: the EWCS separates correct from hallucinated final candidates with an area under the ROC curve of **0.932–0.943** (Appendix D.5). Much of that discrimination is inherited from contradictions, however: hallucinated candidates cluster near 0.3, the score left when the access point is contradicted, which is the same contradiction the safety constraint acts on. Nor is the score calibrated: candidates scoring 0.85 or more were correct only 66% of the time, less often than the flagged band (72%). To test whether any result depends on the score, we re-scored the final candidate of every task under the five schemes planned in Progress Report Two, and with the uncertainty gate removed (Table 6). The re-scoring is exact: under the scheme used it reproduces every recorded decision.

Table 6**:** Sensitivity analysis: outcomes on the final verified candidate (three runs pooled, 2,970 task-runs).

Scheme	Weights L1–L6	Thresholds	Deliver / flag	Abstain	Hallucinations delivered	Correct delivered
A. Proposed (used)	3,3,4,2,1,2	0.85 / 0.60	166 / 1,658	1,146	59	1,513
B. Equal weights	1,1,1,1,1,1	0.85 / 0.60	212 / 1,612	1,146	59	1,513
C. Endpoint-heavy	2,2,6,2,1,2	0.85 / 0.60	1,072 / 752	1,146	59	1,513
D. Strict thresholds	3,3,4,2,1,2	0.90 / 0.70	0 / 1,824	1,146	59	1,513
E. Lenient thresholds	3,3,4,2,1,2	0.80 / 0.50	1,072 / 752	1,146	59	1,513
F. Gate removed (exploratory)	3,3,4,2,1,2	0.85 / 0.60	166 / 1,919	885	**224**	1,579

Under all five planned schemes the withheld tasks, the hallucinations delivered and the correct code delivered are identical; weights and thresholds only move tasks between *deliver* and *deliver with flag*. Removing the gate would instead deliver 261 more candidates, 165 of them hallucinations. This is an architectural finding: **in this design the confidence score is informative, but it does not decide release; the hard verification constraints do.** The EWCS explains delivered code, the safety constraint and uncertainty gate provide the protection, and the decision policy guarantees that the score can never override them. Tuning weights or thresholds would not change what is withheld; changing what the gate treats as unverifiable would.

8.5 Correction, protocol and cost

Told which claim was contradicted and which operations were closest, the model repaired a hallucinated first candidate in 121–150 tasks per run, and about as often produced another hallucination, which was withheld (Appendix D.7). It often bent the protocol (only 53–54% of tasks followed the prompt exactly), but **no task in any run showed the user code the tool had not approved**, which is the practical argument for keeping the delivery decision in the tool. Every prompt cost more (paired Wilcoxon $p < 10^{-160}$ in all 18 comparisons): about 6.2–6.6 times the time, 5.4–6.3 times the energy and 40–46 times the tokens. The tool itself takes 2.3–9.6 s per prompt; the cost comes from the conversation re-sending its growing context. About half the agent's time went to tasks whose final candidate was hallucinated and withheld. The electricity is about one cent per verified prompt; the real price is latency (Appendix D.9).

9 Statistical Analysis

9.1 Primary and secondary hypotheses

McNemar's test depends only on the discordant pairs: tasks where the baseline hallucinated and the agent did not (b), and the reverse (c) (McNemar, 1947). We report 95% intervals from a bootstrap over tasks and a cluster bootstrap over the 141 operation families, which does not assume tasks are independent.

Table 7: Primary test: class A code reaching the user, baseline versus Confident Agent (990 paired tasks per run).

Run	Baseline	Agent	Difference	95% CI (tasks)	95% CI (families)	b / c	Holm p
Q4_K_M	51.8%	1.6%	−50.2 pp	[−53.3, −47.0]	[−63.2, −34.6]	505 / 8	1.7×10^{-137}
Q6_K	50.8%	2.3%	−48.5 pp	[−51.7, −45.3]	[−62.1, −32.7]	490 / 10	1.5×10^{-130}
Q8_0	55.2%	2.0%	−53.1 pp	[−56.3, −50.0]	[−65.7, −37.9]	529 / 3	1.1×10^{-152}

H1 is supported in each run (Holm-adjusted $p < 10^{-129}$; even the family-level intervals exclude zero by more than 30 points). **H2 is supported**: correct code reaching the user rose by 13.5–26.0 points (family-level intervals above +7.7 points; $p \leq 6.4 \times 10^{-20}$). Class A or B code reaching the user fell from 56.5–61.0% to 9.4–11.1%, although class B alone did not fall (Section 8.3). **H3 is supported** (AUC 0.93–0.94) and **H4 is quantified** (Section 8.5; secondary results in Appendix D.11).

9.2 What the significance tests do and do not show

Each p-value is a within-run test on 990 paired tasks. Because the runs repeat the same tasks (Section 7.1), the three significant results are **not three independent confirmations**, and the 2,970 pairs are not independent observations. Holm's correction controls false positives across the three tests but does not make them independent evidence; what the runs show is that the effect survives a change of quantization. The family-level intervals, four to five times wider than the task-level ones, show how much the related prompts within a family matter, and they still exclude zero. Repeat runs at identical settings changed the delivered or withheld status of 11–14% of tasks while reproducing the aggregate rates within about two points (Appendix D.10), so the conclusions rest on aggregate paired

comparisons. Finally, the tests speak for tasks like ours, for this model, on these six APIs; generalising further rests on argument, not on these tests.

9.3 Robustness checks

The primary result holds with the pilot tasks included (all $p < 10^{-130}$); with every undetermined candidate counted as a hallucination or as correct before the review (rates moved by at most 1.5 points); for hallucinations other than a wrong host (−74% to −79%); under all five planned weighting schemes; and in the superseded first series of runs (within about two points).

10 Threats to Validity

We follow the classification of Wohlin et al. (2012); Appendix B.11 summarises the threats and their mitigations in one table.

10.1 Construct validity

- **Hallucination means contract violation**, not rejection by a live service: some APIs silently ignore unknown parameters, so class A code may still return data, but it does not do what it claims. **Withholding is not answering**, so we report correct code delivered, abstention and incorrect rejection beside the hallucination rate.
- **Ground truth is partly researcher-authored.** Three of the six specifications are curated partial contracts, and the planned two-reviewer audit of a 100-task sample of the answer key was not completed. The evaluator's self-check (1,000/1,000 reference solutions correct) and the documented review of every case the curated specifications left open reduce, but do not remove, this threat.
- **The EWCS is evidence coverage, not probability**, and the right request is not a correct program (Section 8.3).

10.2 Internal validity

- **A combined intervention.** The stage decomposition locates the reduction at verification but is not an ablation, so the contribution of each component is not established (Section 8.2). Each condition also generated its own code, so answers differ by sampling as well as treatment; effects far larger than the two-point run-to-run variation limit this threat.
- **Model-controlled protocol.** The model deviated from the prompt in 46–47% of tasks, but the tool's decision on the last verified candidate determines what reaches the user, so deviations affect cost, not the primary outcome.
- **Evaluator bias.** The same team, with AI assistance, built the agent and the evaluator. We separated them by code (no shared modules), by method (sandboxed execution versus static analysis) and by data (only the evaluator reads the answer key), and validated the evaluator on the reference solutions. The planned stratified hand audit of 50 decided candidates per mode was not completed.
- **Other threats.** One verifier rule was changed after run 1 (disclosed; it made the verifier stricter); the manual review was AI-assisted, with a human second review of the 46 decisive

cases (κ = 0.88); and LM Studio's runtime version, which updated itself during the project, is not recorded in the run manifests.

10.3 External validity

The results come from one model (Qwen3.5-9B), six popular public APIs (two of them fake-data services the model knows unusually well, one unusually new), Python with `requests`, single requests and answerable tasks, on one machine and one electricity tariff. Larger models, private APIs, other languages and clients, multi-step workflows and tasks that should be refused may behave differently. The approach also requires a frozen specification for every API it verifies; for an unknown host it always abstains.

10.4 Conclusion validity and reproducibility

The observations are clustered and each task was sampled once per condition, so conclusions rest only on paired aggregate tests within each run, with exact tests and family-level intervals, and subgroup and quantization differences are descriptive (Section 9.2). All inputs are frozen and hashed, every run is stored in full, and every statistic, table and chart is regenerated by scripts in the submission (Appendix J). The model's unseeded sampling cannot be reproduced exactly, so a rerun would reproduce the aggregate results within run-to-run variation rather than task by task.

11 Conclusion

We asked whether an agent that verifies the REST API claims in its own code against frozen, authoritative specifications can stop API hallucinations from reaching the user, and at what cost. For the combined Confident Agent architecture (prompt, verification, one correction and abstention), judged by an independent evaluator on a frozen benchmark, the answer is yes. In each of three runs on the same 990 tasks, hallucinated code reaching the user fell from 50.8–55.2% to 1.6–2.3% (exact McNemar, Holm-adjusted $p < 10^{-129}$), and correct code reaching the user rose from 24.9–36.9% to 50.4–51.5%. The hypothesis of the brief is supported for this architecture, this model and these APIs. Five findings qualify the headline:

1. **The model writes the API it was trained on**: retired and extrapolated versions and similar services. These errors are invisible to the model and obvious to a pinned specification.
2. **We demonstrated the combined architecture, not each component.** The agent's first candidates, written under its prompt, hallucinated almost as often as the baseline (46.7–48.9% against 50.8–55.2%); naming the current hosts changed which hallucination the model made, not whether it made one. The reduction coincides with verification and release, but without an ablation the separate contributions of prompting, verification, correction and abstention are not established.
3. **Release is governed by explicit constraints, not by the confidence score.** The EWCS discriminates well (AUC 0.93–0.94) but measures evidence coverage, is not calibrated, and decides only whether delivered code carries a flag. The safety constraint and the uncertainty gate decide what is withheld: every planned weighting scheme withheld the same code, and removing the gate would have let almost four times as many hallucinations through. The decision policy orders the two so that the score can never override a constraint.

4. **Safe is not useful, and conformant is not correct.** The agent withheld code for 38–40% of tasks and for almost every REST Countries task, where it offers safety but no answer. Passing verification shows conformance to the API contract, not that the code does what the user intended: task-fidelity errors did not fall, and only about one task in ten received code that met every check.
5. **Trust has a price**: about six times the time and 40–46 times the tokens. Locally that is about a cent of electricity per prompt; on a pay-per-token cloud model the tokens would dominate.

Future work follows from these limits: an ablation of the prompt and the tool; binding representative inputs during verification; task-level checks of the intended operation; a shorter conversation to cut cost; completing the human review of the answer key; seeded samples and other models; tasks that should be refused; and monitoring frozen specifications for drift.

A Confident Agent, the brief says, knows when it should not be trusted. Our agent is wrong far less often than the model it wraps, and when it cannot establish that its code is right it says so instead of handing it over. That behaviour does not come from the model becoming more humble, nor from a well-chosen confidence threshold, but from explicit constraints in the architecture that decide what may be released: trust by design rather than by declaration (Vorster, 2026b).

AI Disclosure

In line with the project description's academic-integrity requirements (University of KwaZulu-Natal, 2026, §A.4), we acknowledge the following use of generative AI.

- **Design discussions and early drafts.** DeepSeek and ChatGPT were used to explore the API domain, draft early dataset ideas and discuss verifier designs, including the trade-off between dynamic specification discovery and frozen evidence. The conversation transcripts are retained.
- **Software.** Claude (Anthropic), used through Claude Code, assisted in writing, testing and debugging the dataset builder, the Confident Agent toolbox, the benchmark harness, the evaluator, the statistics compiler and the analysis and diagram scripts.
- **Review.** The first manual review of the 396 undetermined candidates was AI-assisted (Claude), with every rule and decision recorded; a group member independently reviewed the 46 decisive candidates (Section 7.3).
- **This report.** The report was drafted with AI assistance (Claude) from the group's code, data, evaluation plan, earlier reports and the lecturer's feedback, and was condensed and revised, again with AI assistance, in response to the lecturer's feedback on the first draft; the stage analysis of Section 8.2 was added in that revision. Every number, table and chart is generated by scripts in the submission from the recorded experimental data.

All research decisions, including the domain, the hypotheses, the evaluation plan and its changes, were discussed and taken by the group, and the group takes responsibility for the content of this report.

References

Barron, F.H. and Barrett, B.E. (1996) 'Decision quality using ranked attribute weights', *Management Science*, 42(11), pp. 1515–1523.

Cheng, J., Marone, M., Weller, O., Lawrie, D., Khashabi, D. and Van Durme, B. (2024) 'Dated data: Tracing knowledge cutoffs in large language models', *Conference on Language Modeling (COLM)*. arXiv:2403.12958.

Cohen, J. (1960) 'A coefficient of agreement for nominal scales', *Educational and Psychological Measurement*, 20(1), pp. 37–46.

DummyJSON (2026) *DummyJSON documentation: products, users, posts, todos*. Available at: https://dummyjson.com/docs (Accessed: 21 September 2026).

Efron, B. and Tibshirani, R.J. (1993) *An Introduction to the Bootstrap*. New York: Chapman & Hall.

eThekwini Municipality (2026) *Electricity account for the test site, Durban Central, September 2026: energy charge R3.5747 per kWh excluding VAT*. Durban: eThekwini Municipality.

Fielding, R.T. (2000) *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis. University of California, Irvine.

GitHub (2026) *GitHub REST API description (api.github.com OpenAPI document)*. Available at: https://github.com/github/rest-api-description (Accessed: 21 September 2026).

Holm, S. (1979) 'A simple sequentially rejective multiple test procedure', *Scandinavian Journal of Statistics*, 6(2), pp. 65–70.

Huang, L. et al. (2025) 'A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions', *ACM Transactions on Information Systems*, 43(2). arXiv:2311.05232.

Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y.J., Madotto, A. and Fung, P. (2023) 'Survey of hallucination in natural language generation', *ACM Computing Surveys*, 55(12), Article 248.

JSONPlaceholder (2026) *JSONPlaceholder guide*. Available at: https://jsonplaceholder.typicode.com/guide/ (Accessed: 21 September 2026).

Landis, J.R. and Koch, G.G. (1977) 'The measurement of observer agreement for categorical data', *Biometrics*, 33(1), pp. 159–174.

LibreHardwareMonitor (2026) *LibreHardwareMonitor: open-source hardware monitoring*. Available at: https://github.com/LibreHardwareMonitor/LibreHardwareMonitor (Accessed: 27 September 2026).

Liu, F. et al. (2024) 'Exploring and evaluating hallucinations in LLM-powered code generation'. arXiv:2404.00971.

LM Studio (2026) *LM Studio: local large language model server*. Available at: https://lmstudio.ai (Accessed: 30 September 2026).

McNemar, Q. (1947) 'Note on the sampling error of the difference between correlated proportions or percentages', *Psychometrika*, 12(2), pp. 153–157.

Microsoft (2026) *Pyright: static type checker for Python*, version 1.1.414. Available at: https://github.com/microsoft/pyright (Accessed: 25 September 2026).

Open-Meteo (2026) *Open-Meteo OpenAPI specifications (forecast, air quality, marine)*. Available at: https://github.com/open-meteo/open-meteo/tree/main/openapi (Accessed: 21 September 2026).

OpenAPI Initiative (2021) *OpenAPI Specification, version 3.1.0*. Available at: https://spec.openapis.org/oas/v3.1.0 (Accessed: 21 September 2026).

Patil, S.G., Zhang, T., Wang, X. and Gonzalez, J.E. (2023) 'Gorilla: Large language model connected with massive APIs'. arXiv:2305.15334.

Qwen Team (2026) *Qwen3.5-9B* [GGUF model weights, quantizations Q4_K_M, Q6_K and Q8_0]. Distributed through LM Studio (Accessed: 30 September 2026).

REST Countries (2026) *Countries API v5 documentation*. Available at: https://restcountries.com/docs/countries (Accessed: 21 September 2026).

Spracklen, J. et al. (2025) 'We have a package for you! A comprehensive analysis of package hallucinations by code generating LLMs', *34th USENIX Security Symposium*. arXiv:2406.10279.

Stillwell, W.G., Seaver, D.A. and Edwards, W. (1981) 'A comparison of weight approximation techniques in multiattribute utility decision making', *Organizational Behavior and Human Performance*, 28(1), pp. 62–77.

Swagger (2026) *Swagger Petstore: OpenAPI 3 sample*. Available at: https://petstore3.swagger.io/api/v3/openapi.json (Accessed: 21 September 2026).

The Skeptics (2026a) *COMP301 Confident Agent: Progress Report One*. Unpublished group report, University of KwaZulu-Natal.

The Skeptics (2026b) *COMP301 Confident Agent: Progress Report Two, static verification of API hallucinations in LLM-generated code*. Unpublished group report, University of KwaZulu-Natal.

University of KwaZulu-Natal (2026) *COMP301 (2026) Software Design. Confident Agent: Designing for Trust under Uncertainty*. Group project assignment, version 1.0. School of Mathematics, Statistics and Computer Science.

Vorster, L. (2026a) *Trust by Design: Architectural Patterns for Confident AI Agents*. COMP301 course notes, University of KwaZulu-Natal.

Vorster, L. (2026b) *Trust by Design, not by Declaration*. COMP301 executive presentation, University of KwaZulu-Natal, 1 October.

Vorster, L. (2026c) Feedback on Progress Report Two to The Skeptics [email]. Personal communication.

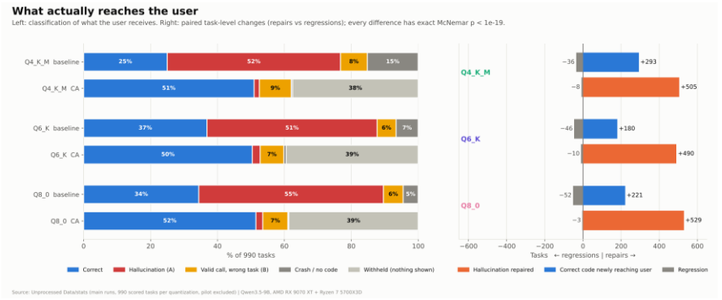
Wilcoxon, F. (1945) 'Individual comparisons by ranking methods', *Biometrics Bulletin*, 1(6), pp. 80–83.

Wilson, E.B. (1927) 'Probable inference, the law of succession, and statistical inference', *Journal of the American Statistical Association*, 22(158), pp. 209–212.

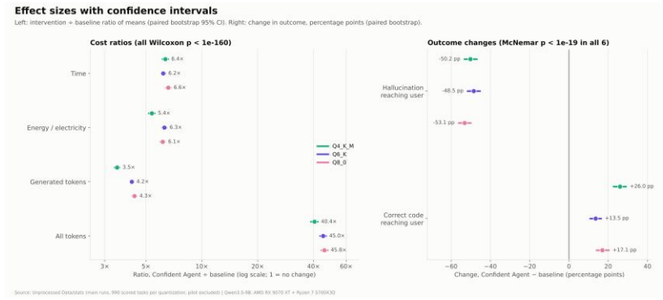
Wohlin, C., Runeson, P., Höst, M., Ohlsson, M.C., Regnell, B. and Wesslén, A. (2012) *Experimentation in Software Engineering*. Berlin: Springer.

Appendix A: Summary of Results

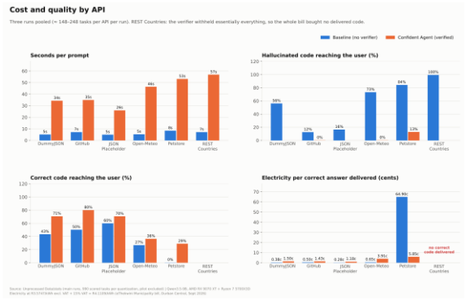
Supplementary charts generated from the unprocessed data (`Findings/Statistical Analysis`). All panels use the 990 scored tasks per quantization (pilot excluded) and price electricity at R4.1109/kWh including VAT. Baseline is blue and the Confident Agent orange throughout.



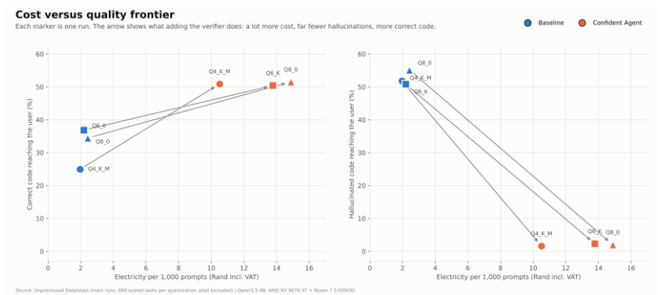
(a) What reaches the user, with paired repairs and regressions



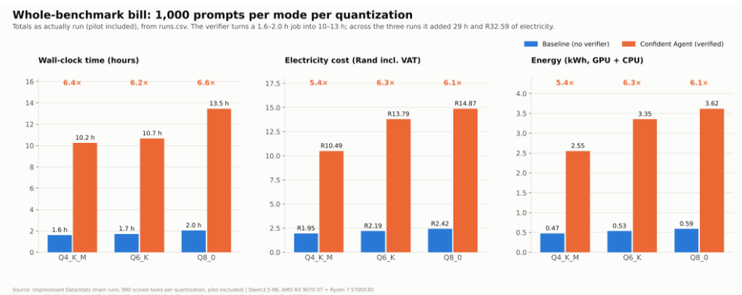
(b) Effect sizes: cost ratios and outcome changes



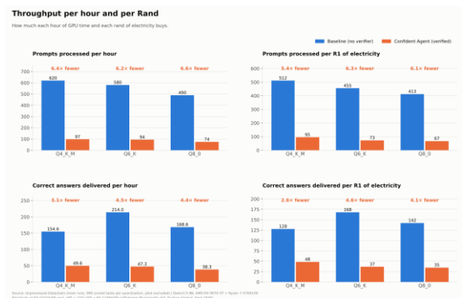
(c) Cost and quality by API



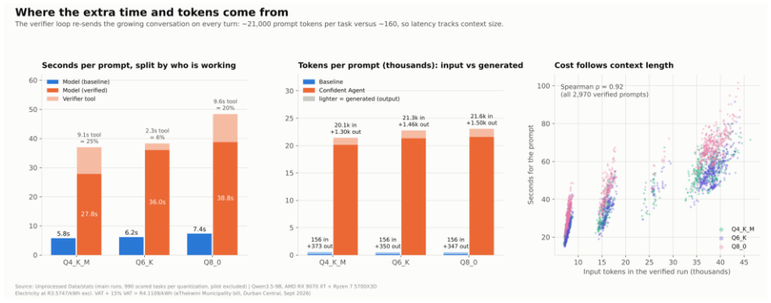
(d) Cost versus quality frontier



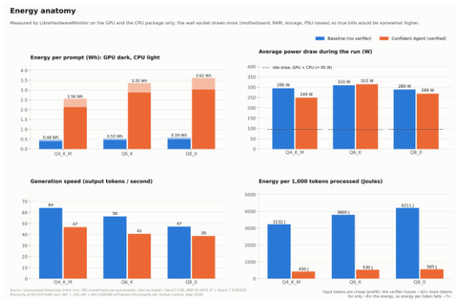
(e) Whole-benchmark time and electricity bill



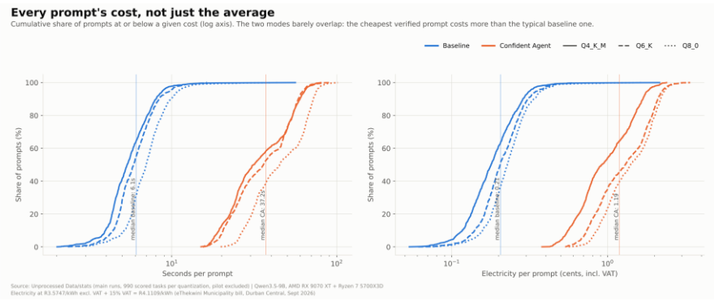
(f) Throughput per hour and per Rand



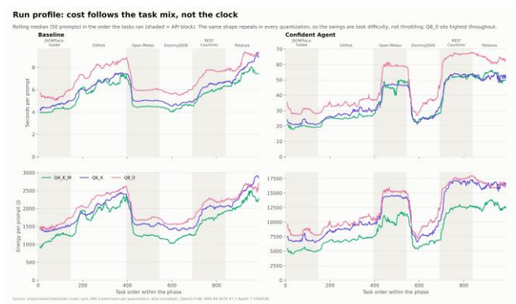
(g) Where the extra time and tokens come from



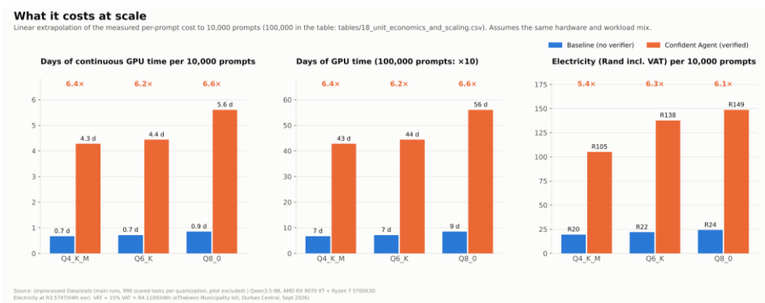
(h) Energy and power anatomy



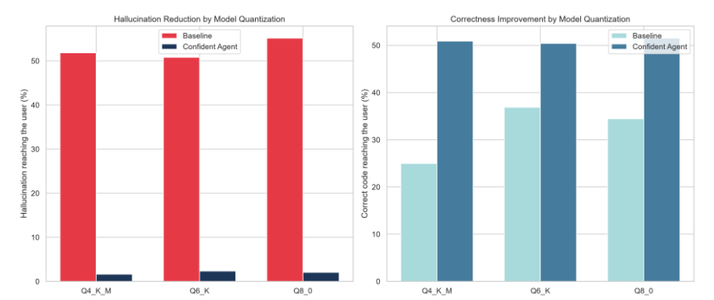
(i) Distribution of per-prompt cost



(j) Run profile: cost follows the task mix



(k) Linear extrapolation to 10,000 and 100,000 prompts



(l) Hallucination and correctness by quantization

Figure 5: Summary of results: supplementary charts (a)–(l).

Appendix B: Benchmark and Methodology Details

This appendix holds the detail behind Sections 3, 4, 5 and 7.

B.1 Mapping to the taxonomy of Progress Report Two

Progress Report Two proposed a ten-way taxonomy (H01–H10). The final evaluator records the same information as *kinds* of class A and class B errors, read from its findings with fixed rules (Appendix H); Table 8 maps the two so that results remain comparable with the earlier design.

Table 8: Mapping of the Progress Report Two taxonomy to the evaluator's error kinds.

Progress Report Two	Evaluator kind (class)	Layer
H01 wrong_endpoint	invented endpoint or method (A); wrong host, version or base path (A); different declared endpoint (B)	Access point
H02 wrong_http_method	undeclared method (A); method not acceptable for the task (B)	Access point
H03 nonexistent_parameter	invented parameter or body field (A)	Input contract
H04 missing_required_parameter / H08 missing_required_field	missing required part or credential (A)	Input / contextual
H05 wrong_parameter_location / H06 wrong_parameter_type / H07 invalid_request_body	invalid value or structure (A); request body differs (B)	Input contract
H09 invalid_header_or_authentication	missing required credential (A); credential malformed (B)	Contextual
H10 nonexistent_api_operation	non-existent `requests` API (A); undocumented response field (A)	Identifier / output

B.2 Evidence and its provenance

- **Representative:** six widely used public REST APIs that span no authentication, API keys, Bearer tokens and OAuth2, official and documentation-derived specifications, and create, read, update, delete and search operations.
- **Objective:** each task carries an exact expected request and its API specification; correctness is decided by comparing requests, not by judging prose.
- **Frozen and hashed:** every provider specification was downloaded on 21 September 2026, stored in the dataset with its SHA-256 hash, and referenced by hash from every task. The task file itself is identified by its hash (`7a09f2bb…`) in the evaluation plan.
- **Safe:** no task contains a real credential; tasks read a placeholder from `inputs["credential"]` and the answer key expects the documented header form (for example `Authorization: Bearer EXAMPLE_CREDENTIAL_NOT_REAL`).
- **Reproducible:** the dataset is generated by a deterministic script from source rules; no language model wrote the reference answers.

The lecturer suggested curating GitHub's large, evolving specification down to the tested operations. We instead froze the **complete** official document (13 MB, SHA-256 `4fbdc7d0…`): a hash freezes a large file as firmly as a small one, the full contract cannot omit an operation by curation error, and it lets

the evaluator distinguish an invented path (class A) from a real GitHub operation that is simply not the one the task asked for (class B). GitHub's prompts pin API version 2026-03-10; Open-Meteo's official specifications were converted from YAML to JSON.

For JSONPlaceholder, DummyJSON and REST Countries no adequate official OpenAPI document exists. Following the lecturer's instruction, we built small project-owned specifications for the operations the benchmark uses, cited the documentation pages they were built from (each page stored with its hash), and marked them as researcher-authored subsets. REST Countries' authentication was verified against its official documentation, which requires a key sent as `Authorization: Bearer <key>` (or `?api-key=`); the answer key records the Bearer form. The service moved to an authenticated v5 interface on a new host (`api.restcountries.com/countries/v5`) and the older public versions are deprecated.

B.3 Task construction and composition

The dataset builder selects documented operations, varies supported optional parameters, request fields and requested response fields, and constructs each record with a checked Python template. Variants that differ only in an identifier or wording do not count as new scenarios. Every record has the seven top-level fields proposed in Progress Report Two (`task_id`, `language`, `natural_language_instruction`, `api_specification`, `ground_truth`, `test_case`, `metadata`). A typical prompt reads:

```
Write a Python function solve(inputs) for REST Countries v5. Retrieve one page of countries.
Use the caller-supplied credential in inputs["credential"], assumed to have the necessary
permissions. Return the decoded JSON response. Return None for a successful empty response.
Raise an exception for an unsuccessful HTTP response. Assume referenced resources and
accounts
exist. Do not execute the function.
```

Table 9**:** Composition of the 990 scored tasks (the 10 pilot tasks are excluded from the analysis).

Dimension	Groups (tasks)	Dimension	Groups (tasks)
Operation	read 494 · create 232 · update 197 · search/filter 37 · delete 30	Specification	official 545 · partial 445
Authentication	none 495 · Bearer 248 · API key 154 · OAuth2 93	Difficulty (heuristic)	easy 267 · medium 720 · hard 3

Difficulty is the dataset's explicit heuristic (number of selected parameters and fields, and whether a credential is needed), not a measured difficulty; with three "hard" tasks it is reported only descriptively.

The API totals match Progress Report Two exactly, but the operation, difficulty and authentication proportions do not match its approximate targets (for example 233 POST tasks against a target of 150, and 30 DELETE tasks against 100). We chose not to relabel tasks to hit percentages: the generator produces the operations each specification genuinely supports, and the audit documents the difference (DATASET_AUDIT.md). The ten pilot tasks overlap the main dataset; results including them are a robustness check (Section 9.3).

B.4 Model and run configuration

Table 10**:** The pinned model and run configuration (identical for baseline and intervention; read from the run manifests).

Setting	Value
Model	Qwen3.5-9B (publisher qwen, architecture qwen35, GGUF), loaded in LM Studio as `qwen3.5-9b-q4`, `-q6`, `-q8`
Quantizations	Q4_K_M, Q6_K and Q8_0 (as reported by LM Studio for the loaded model)
Serving software	LM Studio, OpenAI-compatible server on 127.0.0.1:1234; tool use enabled
Context length	30,208 tokens loaded (model maximum 262,144)
Sampling	temperature 0.2; no fixed seed (one sample per task per run); reasoning ("thinking") off
Limits	max 4,096 completion tokens per reply; max 4 model turns per intervention task; 900 s request timeout
Baseline system prompt	"You are an API coding assistant. Write the Python code the user asks for, using the `requests` library. Reply with the complete code in a single fenced python code block."
Intervention system prompt	Confident Agent prompt for toolbox 3.0.0 (SHA-256 `0ebc4dce`…), operator note removed; tool offered: `verify_candidate` only
Prompts sent	rest_api_prompts_v1_DRAFT.jsonl (SHA-256 `c4c68a64`…), identical in both conditions
Software versions	benchmark 2026-09-27.2; toolbox 3.0.0 (decision policy 2026-09-25.1); Pyright 1.1.414; harness Python 3.13.15; Node.js 24.15.0
Machine	Windows 11; AMD Ryzen 7 5700X3D (8 cores); 32 GB RAM; AMD Radeon RX 9070 XT; Durban Central
Run dates (SAST)	Q6_K 30 Sep–1 Oct; Q8_0 1–2 Oct; Q4_K_M 2–3 Oct 2026

Earlier documents described the local model and its settings inconsistently (for example Qwen3.6 9B in Progress Report One and Qwen3.5 9B-Instruct in Progress Report Two, with different temperatures and context lengths). The final experiment uses exactly this one configuration for both conditions and all three quantization runs.

B.5 Experimental pipeline

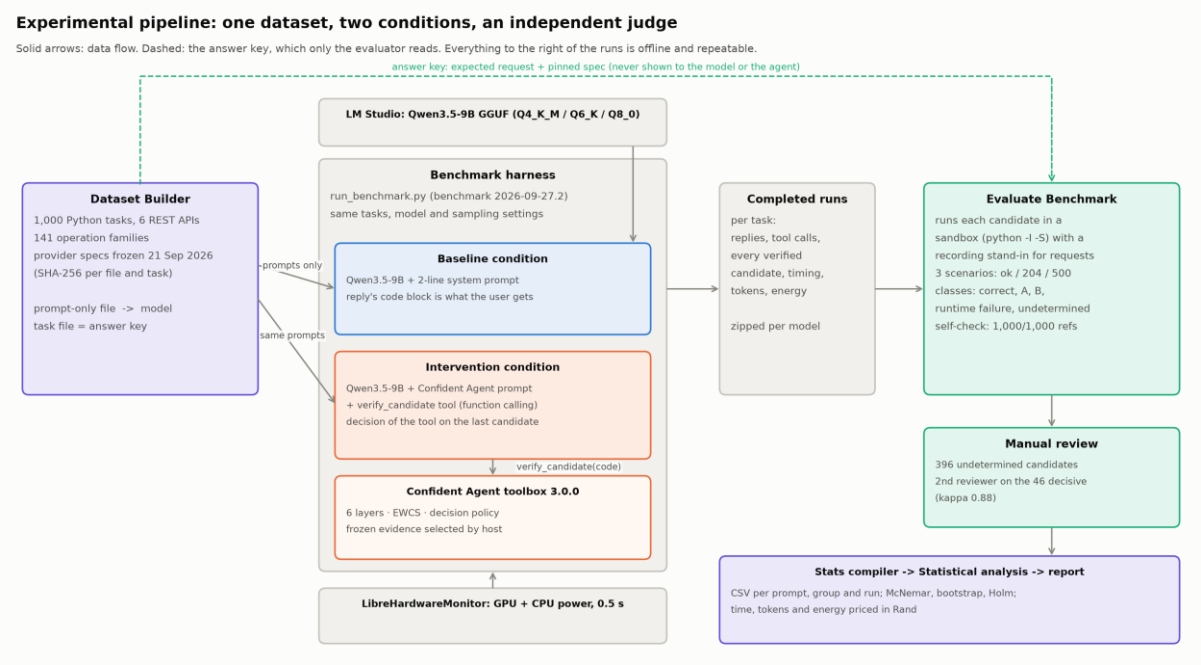


Figure 6**:** Experimental pipeline. The answer key reaches only the evaluator; the model and the agent see the prompt alone.

B.6 Procedure and measurements

The benchmark harness sends each prompt to LM Studio. In the intervention, code shown in the model's reply that the tool did not approve would be recorded as unapproved; this never occurred. Every message, tool call, candidate and timing is saved, and tasks are resumable.

Table 11**:** What was measured for every task and condition.

Measure	Definition
Primary outcome	Class A code reaches the user (1/0), as classified by the independent evaluator
Secondary outcomes	class A or B; any error (A, B or runtime failure); correct code reaches the user; incorrect acceptance and rejection; abstention
Verifier behaviour	decision, EWCS, layer states, corrections, protocol deviations, verification calls, tool failures
Task behaviour	returns the expected value; returns None for an empty 204 response; raises on a 500 error
Time	wall-clock seconds per task, split into model time (LM Studio) and tool time (verification)
Tokens	prompt and completion tokens summed over the task's requests, as reported by LM Studio
Energy	GPU package and CPU package power from LibreHardwareMonitor, sampled every 0.5 s and integrated over the task; idle power measured for 10 s before each run

AMD GPUs offer no command-line power reading on Windows, so the harness reads LibreHardwareMonitor's local web server, running as administrator, for the "GPU Package" sensor of the RX 9070 XT and the CPU "Package" sensor. The measured energy excludes memory, storage, motherboard and power-supply losses, so the true wall-socket energy is higher. Energy is converted to Rand at the eThekwini Municipality tariff on the electricity account for the test site in Durban Central for September 2026, **R3.5747 per kWh excluding VAT**, plus 15% VAT, i.e. **R4.1109 per kWh** (eThekwini Municipality, 2026):

$$Cost\,(R) \;=\; \frac{E\,(J)}{3\,600\,000\,J/kWh} \times R3.5747/kWh \times 1.15 \;=\; E \times 1.1419 \times 10^{-6}\,R/J$$

B.7 The independent evaluator

The evaluator executes each candidate in a separate, isolated Python process (`python -I -S`) with the task's test inputs and a recording stand-in for `requests` that has the real library's names and signatures but sends nothing: sockets, subprocesses and file writes are blocked and each run has a 10-second limit (Figure 7). This yields the exact requests the code would send, including those built in loops and branches, and every response field it reads. The self-check against all 1,000 reference solutions guards against an evaluator that is stricter or looser than the answer key.

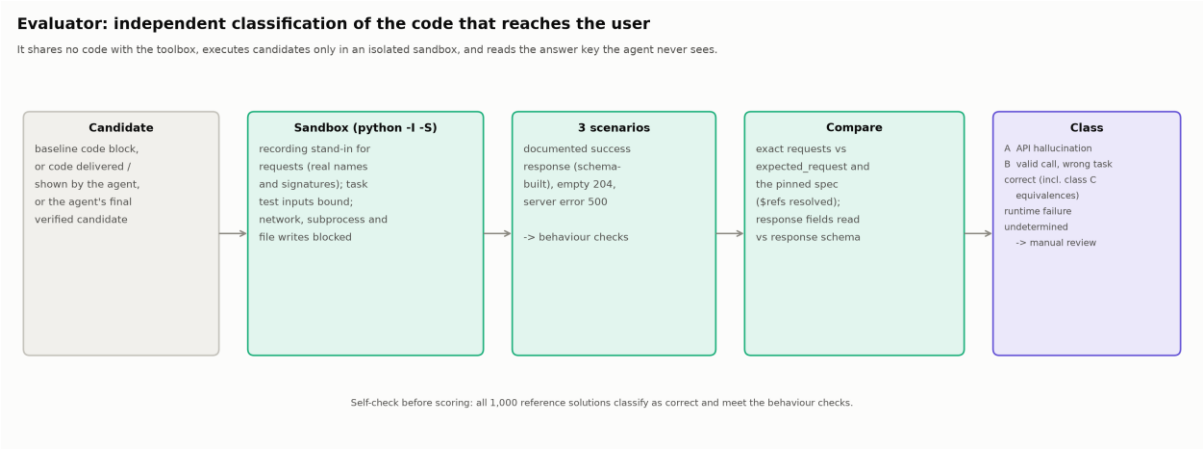


Figure 7: The independent evaluator: sandboxed execution, three response scenarios and comparison with the answer key.

B.8 Manual review of undetermined candidates

Each open question (for example: does DummyJSON support a `sort` query parameter? does REST Countries v5 have a `/v5/name/{name}` path?) was checked once against the provider's documentation and, where the documentation was silent, against the live public API with harmless read requests (for example, `?sort=title` returns exactly the unsorted list, while the documented `?sortBy=title` sorts it). Each answer was recorded as an explicit rule, so that every candidate asking the same question received the same verdict, and the evaluation plan's rules were then applied unchanged: a name the provider does not have is class A; a name it has is judged as for an official specification. The three disagreements between the two reviews were JSONPlaceholder candidates that read a resource from a list endpoint (`/comments?id=…` instead of `/comments/{id}`); the group settled them as correct because they return the requested resource, which relaxes the closed equivalence list for those three cases. All 396 decisions, with their reasons, are in the submission (`reviews.csv`, `review_rules.py`).

B.9 Changes to the evaluation plan

Table 12: Changes to the evaluation plan after it was fixed on 27 September 2026.

Date	Change	Reason and effect
27 Sep	Undeclared request-body fields become unverifiable in the verifier	Found by inspecting run 1 (8 hallucinated fields delivered); disclosed; applies to all reported runs
30 Sep	Holm family limited to the main runs	Decided before those runs were scored

Date	Change	Reason and effect
6 Oct	Findings use the final series (Q4_K_M, Q6_K, Q8_0, 30 Sep–3 Oct) instead of run 2 and the first Q6_K/Q8_0 runs	Machine throughput halved mid-way through the first Q6_K run (GPU power 240 W to 193 W), invalidating its cost figures; accuracy agrees with the first series within about 2 points (Appendix D.10)
6 Oct	Manual review made with AI assistance, not by two independent members	Time constraints; documented rules and reasons for all 396 decisions
7 Oct	Second, human review of the 46 decisive candidates; three relaxed to correct	κ = 0.88; changes correct-code figures by one task (Q6_K) and two (Q8_0)

B.10 Statistical methods

Table 13**:** Statistical methods (evaluation plan §6).

Question	Method
Does the intervention change the primary outcome?	Exact two-sided McNemar test on the paired binary outcome within each run (McNemar, 1947); α = 0.05
How large is the change?	Difference in proportions with 95% percentile bootstrap intervals (10,000 resamples) over tasks and, to allow for related prompts, over the 141 operation families (Efron and Tibshirani, 1993)
Three runs, one hypothesis	Holm's step-down correction across the three runs (Holm, 1979); the runs repeat the same tasks and are not independent replications
Per-group rates	Wilson 95% intervals (Wilson, 1927); subgroup tests are descriptive
Cost	Wilcoxon signed-rank test on paired per-task costs (Wilcoxon, 1945) with rank-biserial effect size; Kruskal–Wallis across quantizations
Confidence metric	Area under the ROC curve, correct versus class A final candidates; share correct per EWCS band
Agreement	Cohen's κ for the two reviewers, between repeated runs and between quantization runs (Cohen, 1960)
Robustness	Pilot tasks included; undetermined counted as A and as correct; hallucinations other than wrong host; the five planned weighting schemes

B.11 Summary of threats to validity

Table 14**:** Summary of the main threats to validity and their mitigation.

Threat	Risk	Mitigation
Combined intervention	Effect attributed to verification is partly prompting	Stage decomposition (Section 8.2); non-host hallucinations; stated as not an ablation
Evaluator built by the same team	Correlated blind spots	Separate code and method; sandboxed execution; self-check on 1,000 references
Curated partial specifications	Ground truth errors	Labelled; undetermined cases reviewed against documentation and live APIs; κ = 0.88
Answer-key review incomplete	Systematic answer-key errors undetected	Self-check; review of open cases; disclosed as a limitation
Related observations	Overstated precision or independence	Tests within each run; cluster bootstrap over 141 families; runs treated as robustness, not replication

Threat	Risk	Mitigation
One sample per task	Noisy task outcomes	Paired aggregate tests; repeat-run agreement reported
Weights and thresholds chosen by us	Result depends on arbitrary numbers	Five pre-planned schemes: no change to withheld or delivered outcomes
Runtime drift	Unequal conditions between runs	Final series on consecutive days; first series reproduces accuracy within ~2 points
One model, six APIs, Python	Limited generality	Stated scope; design is API-agnostic but evidence-dependent

Appendix C: Architecture Details

This appendix holds the detail behind Section 6.

C.1 The Trust-by-Design patterns

Table 15**:** How the Confident Agent realises the Trust-by-Design patterns (Vorster, 2026a).

Pattern	Realisation in the Confident Agent
Independent Verification	The model proposes code; a deterministic tool (claim extractor, Pyright, OpenAPI matcher) judges it. The verifier never asks the model whether it is right.
Domain-Specific Validation	Evidence is the API's own contract: OpenAPI paths, parameters, schemas and security schemes, plus the Python package registry and type checker.
Evidence Fusion	Six layer verdicts, each with an evidence tier, are combined by the EWCS; missing and contradictory evidence are kept apart (unverifiable versus verified incorrect).
Confidence Gating	The decision policy releases code only if no layer is contradicted, no critical check is unresolved, and the EWCS clears a threshold; it can deliver, flag, request a correction, or abstain.
Deferred Commitment	A contradicted candidate is not released; the agent may rewrite it once and must have the new code verified. An abstention without a contradiction is final.
Provenance Preservation	Every verification records the candidate's SHA-256, the evidence file and its hash, per-layer evidence details and the decision; the report renderer refuses code whose hash differs from the verified candidate.

C.2 Where the tool attaches, and why

Table 16**:** Options for attaching verification to the model.

Option	Advantages	Disadvantages
A. External post-hoc controller (CLI): generate, then verify and correct outside the model	Full enforcement; deterministic; the model cannot skip a step	Not how developers use assistants; corrections need a second prompt the model did not plan for
B. In-loop tool via function calling (chosen)	The deployable artefact: a model with a prompt and a tool, as in LM Studio; the model sees the evidence and can correct itself	The model controls the protocol and may deviate; deviations must be measured
C. MCP server (Progress Report Two)	Portable across MCP clients	Adds a server process; LM Studio's tools-provider SDK gives the same function-calling interface natively
D. Multi-agent pipeline (Progress Report One)	Specialist agents per check	Each agent is another model that can hallucinate; deterministic checks are cheaper and stronger evidence

Option A survives as the toolbox's external controller and pipeline scripts, which we used for engineering checks of the verifier. Option C would be a packaging change, not a design change, and is left for future work. Option D was rejected on evidence grounds: the Trust-by-Design notes caution against using a model as judge where a deterministic checker exists (Vorster, 2026a).

C.3 Claim extraction

The extractor follows variables, f-strings, concatenation, dictionaries built key by key, `if`/`try` branches and helper functions that return `response.json()`. Each claim records its type (import, method call, endpoint, input field, output field), its line and its provenance. When the extractor meets a construct it cannot follow (a `requests.Session`, another HTTP client, a class, async code), it reports an issue that names the layers it affects, and only those layers are downgraded to unverifiable; early versions downgraded every layer and silently scored most valid code as unverifiable (Appendix C.8).

C.4 The six layers in detail

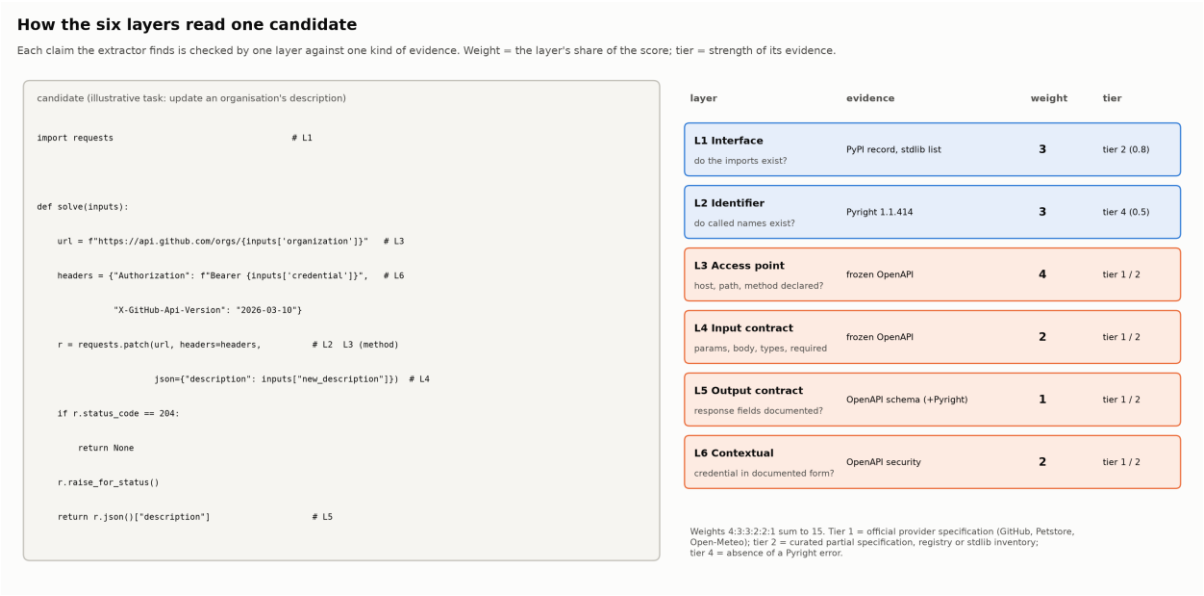


Figure 8: How the six layers read one candidate: each claim is checked by one layer against one kind of evidence.

- **L1 Interface.** Every imported module must exist: standard-library modules against the pinned interpreter's module list, third-party packages against a PyPI record (frozen for `requests` in the experiment so that a network timeout cannot change a verdict).
- **L2 Identifier.** Pyright 1.1.414 in standard mode type-checks the exact candidate. An undefined name, attribute or call is a contradiction; argument-type errors count only on API lines. Absence of an error is weak evidence (tier 4).
- **L3 Access point.** The host and base path must match a server of the frozen specification, and the path template and HTTP method must match a declared operation. A mismatch is a contradiction whose evidence names the closest declared operations, which the agent can use to correct. An unknown host has no evidence and makes the layer unverifiable.
- **L4 Input contract.** Required parameters, their locations (path, query, header, body), types and request-body properties are checked against the matched operation. Runtime values are not guessed; an undeclared query parameter or body field is unverifiable unless the schema explicitly allows or forbids extra properties.
- **L5 Output contract.** Every response field the code reads is followed through the success-response schema. A field the schema documents verifies; a field it does not document cannot be verified, and contradicts a schema that excludes it.

- **L6 Contextual.** The operation's security requirements are checked against the actual request: a missing credential or a wrong scheme is a contradiction; OAuth2 scopes, which are granted to a token rather than expressed in code, are reported as a note.

A layer aggregates its claims by taking the most severe verdict, so one contradicted parameter contradicts the layer.

C.5 Frozen evidence, selected by host

An early design discovered specifications dynamically at run time, by probing well-known paths and searching the web. We abandoned it for two reasons recorded in our design discussions and confirmed by the lecturer's feedback: a live specification can change between the baseline and agent runs, which destroys the comparison, and four of the six benchmark APIs publish no discoverable specification, so a dynamic resolver would have measured discovery rather than verification. The toolbox instead bundles the same frozen snapshots as the dataset, each with its provenance and hash, and a registry maps each host to its snapshot.

C.6 The EWCS: signed score, weights, tiers and ceilings

The EWCS lies in [0, 1]. It is 1 only when every applicable layer is verified by authoritative (tier 1) evidence, and it falls both when evidence is weaker and when layers are unverifiable or contradicted. Not-applicable layers leave the denominator, so a candidate is not penalised for having no response fields to read. A diagnostic **signed score** S keeps contradictions visible by using σ_l = +1 (verified correct), −1 (verified incorrect) or 0 (otherwise), so that S lies in [−1, 1]:

$$S = \frac{\sum_{l \in A} w_l \cdot t_l \cdot \sigma_l}{\sum_{l \in A} w_l}$$

The weights are a design hypothesis, fixed before any run and then tested (Section 8.4). They come from a two-step procedure that is standard in multi-criteria decision analysis when only the *order* of importance can be justified: rank the criteria, then convert ranks to weights with the **rank-sum rule** (Stillwell, Seaver and Edwards, 1981; Barron and Barrett, 1996):

$$w_k \propto n - r_k + 1, \; n = 4 \; severity \; ranks \; \Rightarrow \; 4 : 3 : 2 : 1$$

We ranked the layers by the consequence of a hallucination in that layer for the user, judged on two criteria: whether the call can still do anything useful, and whether the failure is silent or loud (Table 17).

Table 17**:** Severity ranking of the layers and the rank-sum weights (total 15).

Rank	Layer	Weight	Why
1	L3 Access point	4	A wrong host, path or method sends the request to a resource that does not exist or to the wrong one; nothing downstream can be right. It is the dominant hallucination kind in our pilot and baseline, and layers 4–6 are only defined relative to the matched operation.
2	L1 Interface, L2 Identifier	3	A non-existent module or function makes the program fail outright. Fatal, but loud: the developer sees an error at once.
3	L4 Input contract, L6 Contextual	2	A wrong parameter or credential usually produces an HTTP error or a partial result. Serious but situational, and frequently loud (4xx responses).

Rank	Layer	Weight	Why
4	L5 Output contract	1	Reading an undocumented field is a downstream data bug; the request itself succeeds.

The tier factor discounts a verdict by the strength of the evidence behind it, on the familiar principle of evidence hierarchies that primary, authoritative sources outrank derived ones and that an absence of contrary evidence is weaker than positive evidence (Table 18).

Table 18**:** Evidence tiers and factors.

Tier	Factor	Sources	Rationale
1	1.0	Provider-published OpenAPI (GitHub, Petstore, Open-Meteo)	The provider's own contract: the strongest evidence available without calling the service.
2	0.8	Curated partial specifications; PyPI record; interpreter module list	One step removed from the authority (transcribed from documentation, or registry metadata); a 20% discount reflects transcription and coverage risk.
3	0.9	Runtime execution (reserved, not used)	Observed behaviour would be stronger than metadata but weaker than the contract it may not exercise. The agent never executes code, so tier 3 never occurs.
4	0.5	Pyright found no error	Absence of a type error is "no evidence against", not evidence for: half credit, the point of indifference.
none	0	No evidence	No evidence earns no credit.

Worked example. For the GitHub candidate of Figure 8, assume all claims verify: L1 by the PyPI record (tier 2: 3 × 0.8 = 2.4), L2 by a clean Pyright run (tier 4: 3 × 0.5 = 1.5), L3, L4 and L5 by GitHub's official specification (tier 1: 4 + 2 + 1 = 7). GitHub's OpenAPI document does not model authentication, so L6 is not applicable and the denominator is 3 + 3 + 4 + 2 + 1 = 13. Hence EWCS = 10.9 / 13 = **0.838**, and the decision is *deliver with flag*. If the same code had called the invented path `/organizations/{org}`, L3 would be verified incorrect, L4 and L5 could not be matched to an operation (unverifiable), and EWCS = 3.9 / 13 = **0.30**; the score is irrelevant, however, because the contradiction triggers the safety constraint. This is why hallucinated candidates cluster at an EWCS of about 0.3 in Figure 15.

Because the weights and tiers are fixed, the highest EWCS each kind of evidence can reach is known in advance (Table 19). The *deliver* threshold of 0.85 was set at the point that requires authoritative evidence for every API layer *including* authentication: only fully modelled official specifications can reach it. Every candidate checked against a curated specification can at most be *delivered with a flag*, which is the honest outcome for researcher-authored evidence. Since a candidate that passes the uncertainty gate has every applicable layer verified, its EWCS is at least that of the weakest row, 0.65, which is why the 0.60 threshold never binds.

Table 19**:** Maximum attainable EWCS when every applicable layer verifies (L1 tier 2, L2 tier 4).

Evidence for layers 3–6	Applicable weight	Max EWCS	Best decision
Official, all of L3–L6 applicable (e.g. Petstore with security)	15	0.860	deliver
Official, no response fields read (L5 n/a)	14	0.850	deliver

Evidence for layers 3–6	Applicable weight	Max EWCS	Best decision
Official, authentication not modelled (GitHub, Open-Meteo)	13	0.838	deliver with flag
Official, L5 and L6 n/a	12	0.825	deliver with flag
Curated partial specification (JSONPlaceholder, DummyJSON, REST Countries)	12–15	0.725–0.740	deliver with flag
No API call (only L1 and L2 applicable)	6	0.650	deliver with flag

C.7 Static and dynamic structure

Figure 9 shows the main types of the verification core and Figure 10 the interaction for one benchmark task.

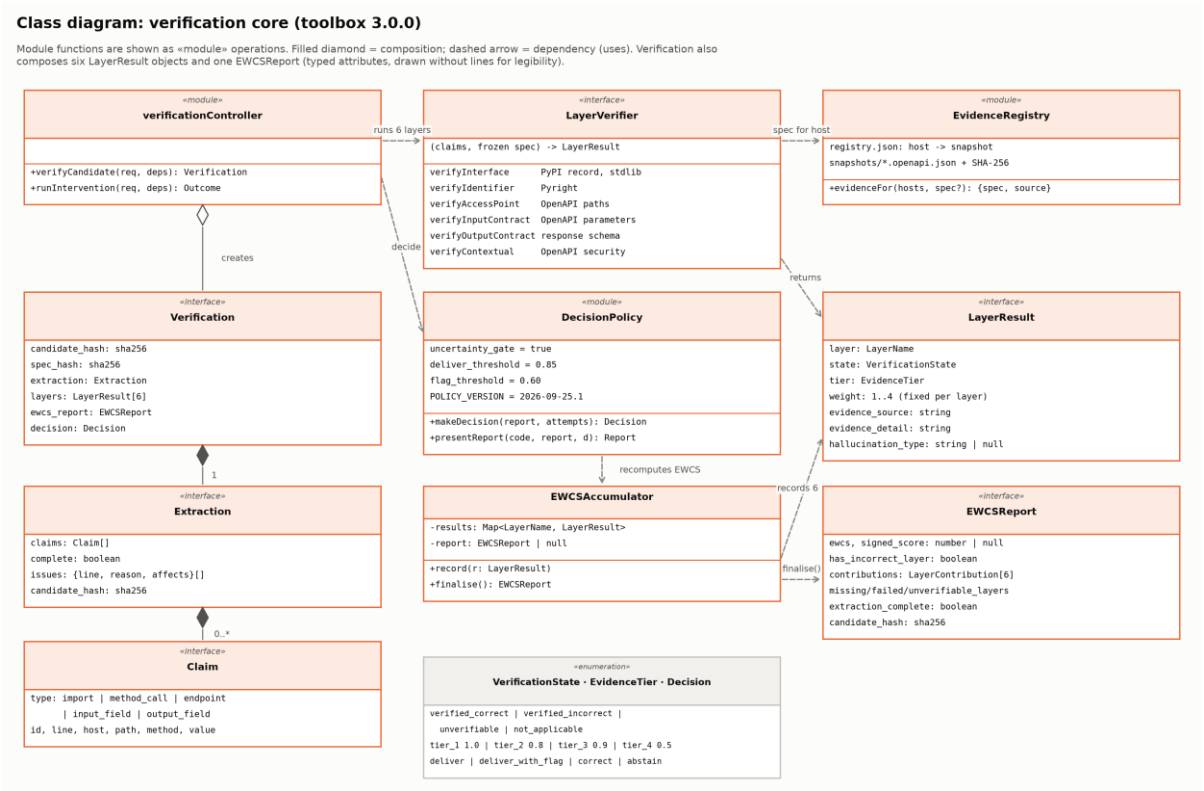


Figure 9**:** Class diagram (UML) of the verification core in toolbox 3.0.0.

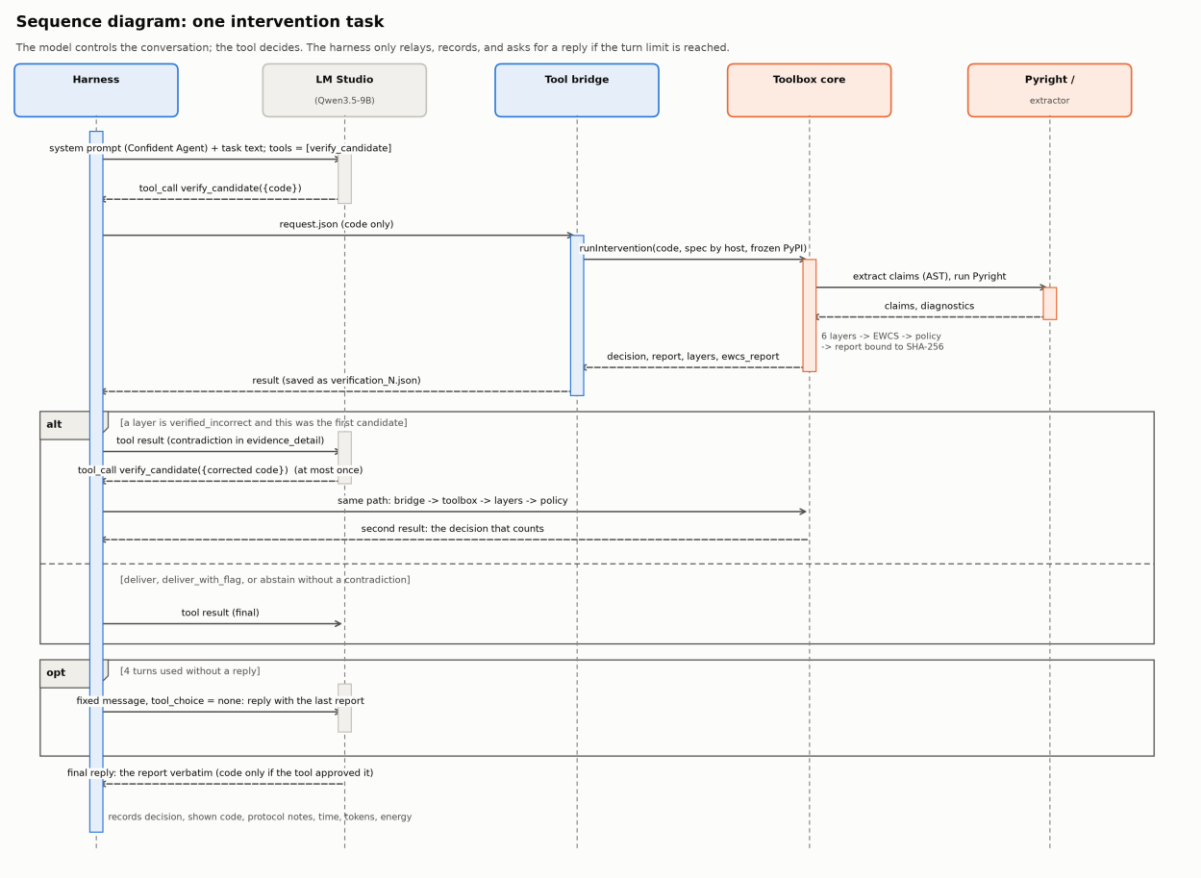


Figure 10**:** Sequence diagram (UML) of one intervention task, including the single permitted correction and the reply request at the turn limit.

C.8 Design evolution and verification of the verifier

The design changed substantially between Progress Report Two and the experiment, and each change was driven by evidence rather than preference. Three changes stand out.

- **From dynamic discovery to frozen evidence** (Appendix C.5): live specifications would have made the comparison irreproducible and the coverage of four APIs zero.
- **From a general-purpose plugin to a standalone toolbox.** The first working version was built into an existing LM Studio plugin that also offered code execution, a terminal, web search, a browser and memory. Those tools would have let the model work around verification and confounded the experiment, and code execution contradicts our safety principle. Toolbox 3.0.0 removes all of them and exposes verification tools only.
- **Thirteen scoring defects found by testing the verifier on known answers.** Running the first implementation on the dataset's own reference solutions (known-correct code) gave a mean EWCS of 0.007 and falsely contradicted valid REST Countries code. The causes were engineering defects, not design flaws: any unsupported construct downgraded every layer, `headers["X"] = v` assignments were invisible, URLs containing a runtime value became wholly unknown, Pyright ran in strict mode, and others. All thirteen were fixed and documented with the decisions they required.

Table 20 summarises the engineering checks of the final verifier. These are checks of the apparatus, not experimental results: they show that it accepts known-valid code and rejects known-invalid code before it was used on model output.

Table 20**:** Engineering checks of the verifier on known-valid and known-invalid code (toolbox reference regression).

Input	Result
1,000 reference solutions (known valid), before the fixes (150-task sample)	mean EWCS 0.007; 138 abstain, 12 false contradictions
1,000 reference solutions, final verifier	mean EWCS 0.784; 99 deliver, 888 deliver with flag, 13 abstain; **0 contradictions**
1,000 mutants with an invented path	all 1,000 contradicted (correction requested)
1,000 mutants with an undeclared method	all 1,000 contradicted
1,000 mutants with a wrong host	all 1,000 abstain (no evidence for an unknown host)

The 13 remaining abstentions on valid code are documented evidence limits (GitHub request bodies declared with `oneOf`, one ambiguous path template).

One change was made after inspecting benchmark run 1, and is disclosed as such in the evaluation plan: an undeclared request-body field is now unverifiable, as an undeclared query parameter already was, because eight run-1 deliveries had sent hallucinated body fields (for example `name` where GitHub's label API declares `new_name`). All findings in this report use the verifier after this change.

Appendix D: Supplementary Results

This appendix holds the supporting analyses behind Sections 8 and 9. All figures use the 990 scored tasks per run after the manual review; pooled figures add up the three runs over the same tasks and are descriptive.

D.1 Scorecard

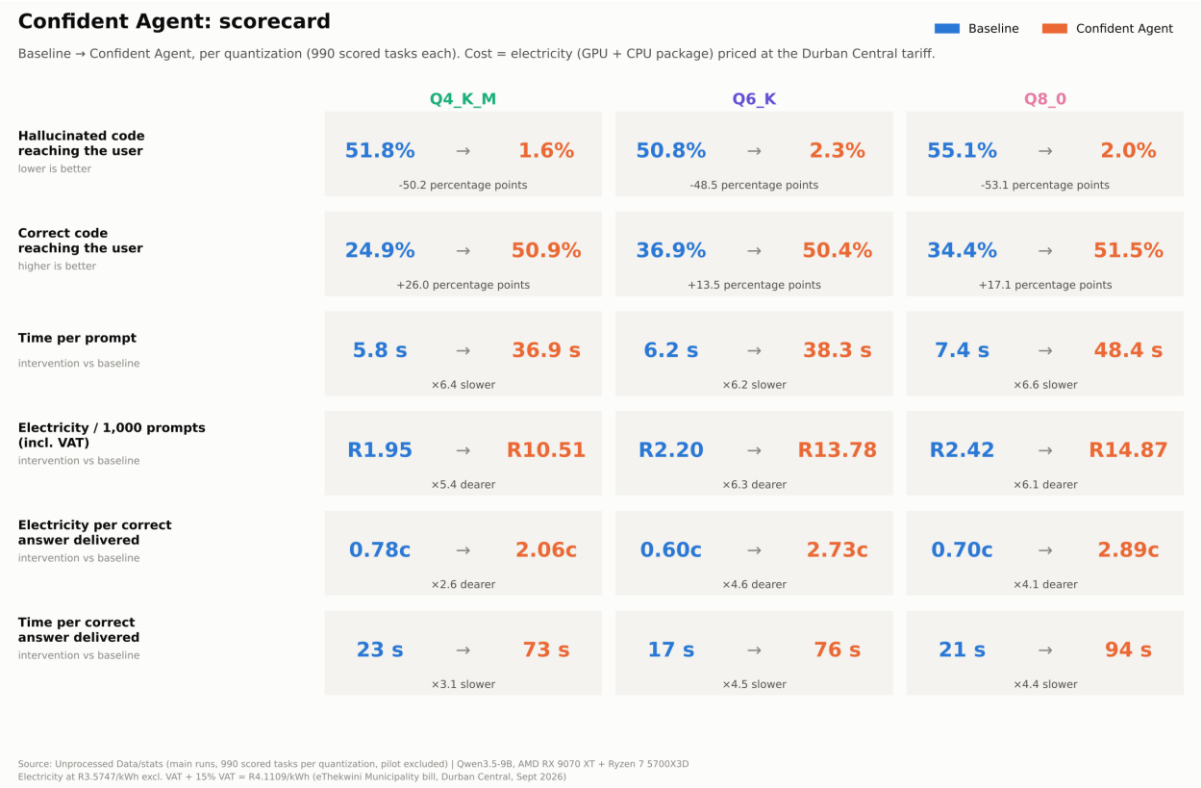


Figure 11**:** Scorecard: baseline versus Confident Agent for each quantization (990 tasks each). Electricity priced at R4.1109/kWh including VAT.

Pooled over the three runs, hallucinated code reached the user in 1,562 of 2,970 task-runs (52.6%) without the agent and in 59 (2.0%) with it, and correct code in 953 (32.1%) and 1,513 (50.9%). The agent's final accuracy is almost the same at every quantization (50.4–51.5% correct) although the baselines differ: the lower-precision model gains most (+26.0 points of correct code at Q4_K_M) because the agent repairs the coding errors in which Q4_K_M's baseline was weakest.

D.2 Results by subgroup

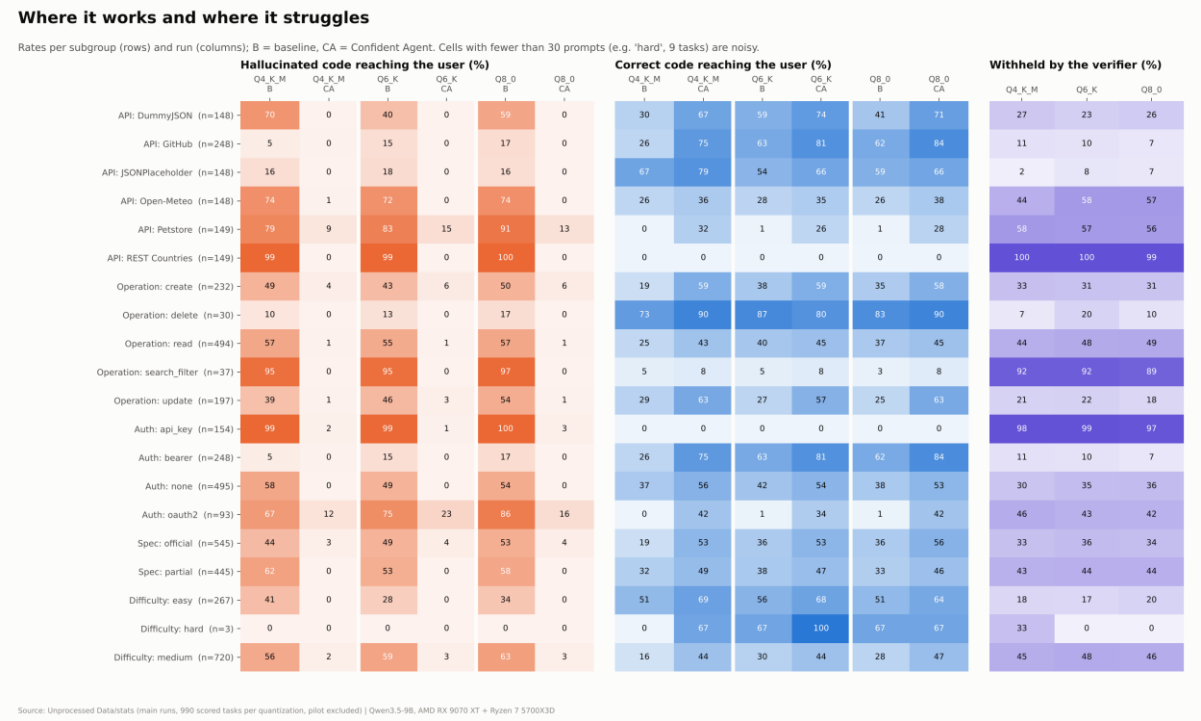


Figure 12**:** Rates per subgroup and run: hallucinated code reaching the user, correct code reaching the user, and code withheld by the verifier (B = baseline, CA = Confident Agent).

APIs the model knows well improve the most: GitHub (Bearer authentication) rises from 26–63% correct to 75–84%, and DummyJSON from 30–59% to 67–74%, with no hallucination reaching the user in any run. Petstore carries almost all the residual hallucinations (9–15% of Petstore tasks), and REST Countries is entirely withheld (Section 8.3).

D.3 Kinds of hallucination

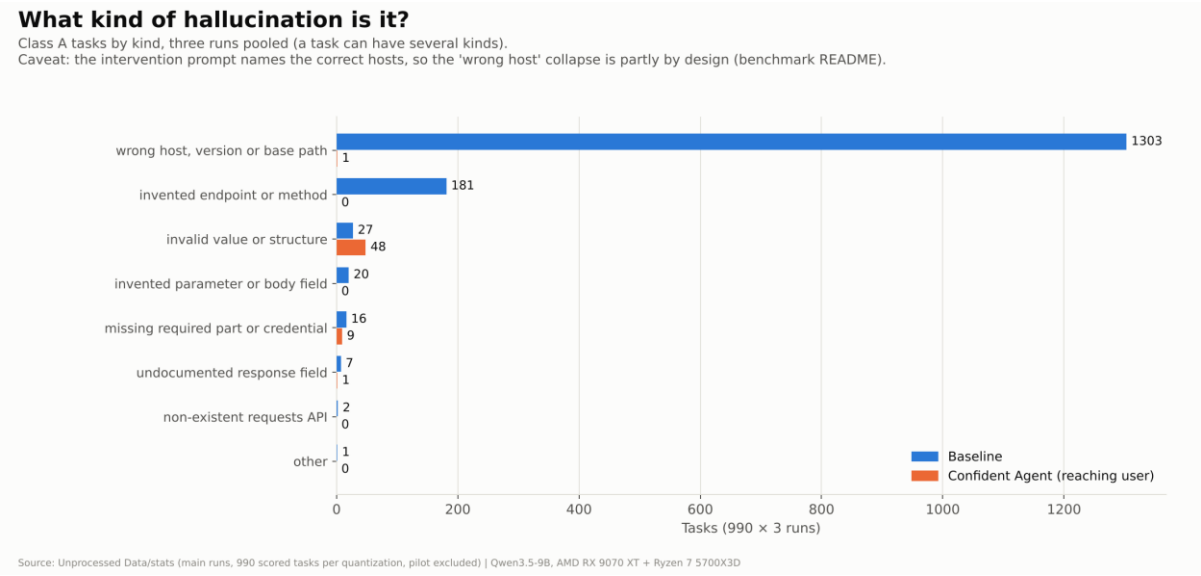


Figure 13**:** Kinds of API hallucination in the code reaching the user (three runs pooled; a task can have several kinds).

Wrong hosts, versions and base paths make up most baseline hallucinations, and they almost vanish in the delivered code. One kind grows: *invalid value or structure*, from 27 to 48 pooled tasks. In 48 of the 59 residual hallucinations a Petstore request body places a value from `inputs` where the schema expects a string (for example `category.name` set to the object `{"id": 1}`), and in 9 more a required `api_key` credential is missing. The cause is a deliberate design decision: the verifier treats values that only exist at run time as opaque, so it cannot see *which* value lands in a field, whereas the evaluator binds the task's test inputs and can. Binding representative inputs during verification, which the extractor already supports, is the most direct improvement to the design.

D.4 The verifier as a hallucination detector

Treating the verifier as a classifier of the agent's final candidate (positive = class A, predicted positive = withheld) gives the confusion matrices of Figure 14. Recall is **94–96%**: it lets very few hallucinations through. Precision is **86–92%**: most of what it withholds really is hallucinated. Its false alarms (20–28 correct candidates withheld per run, plus some wrong-task and crashing candidates) are the incorrect rejections the brief asks us to count. When class B errors are also treated as positives, recall drops to 76–80%: the verifier checks the API's contract, not whether the call is the one the task wanted.

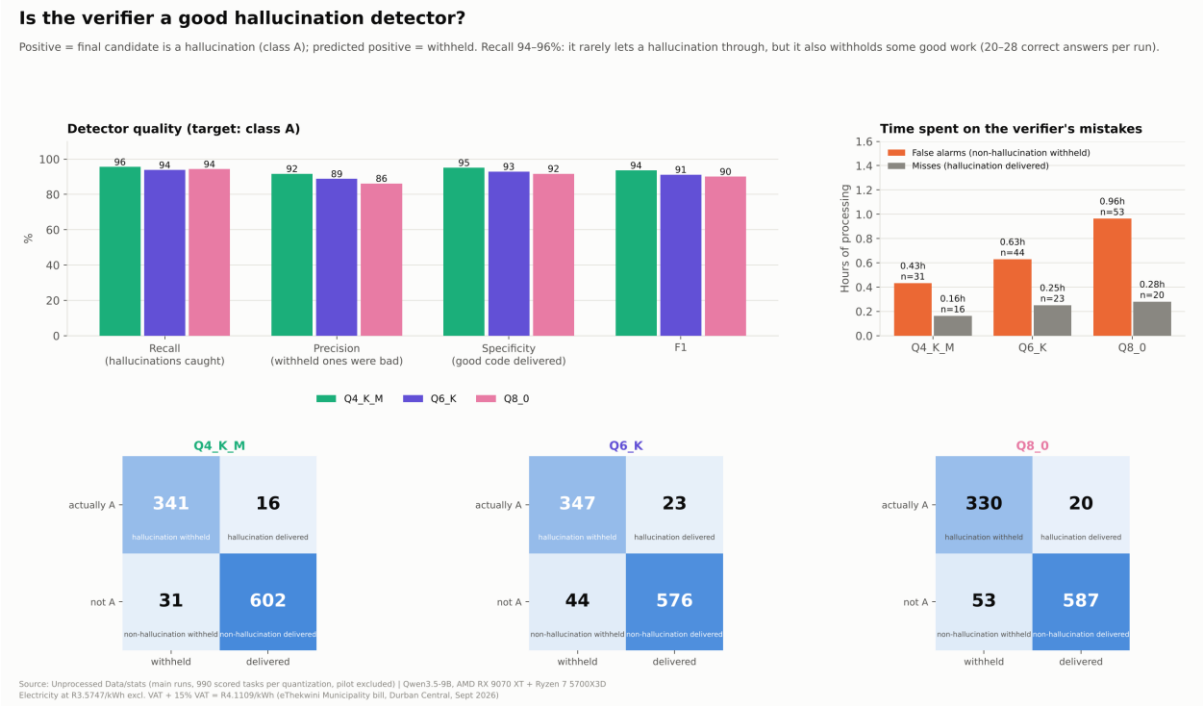


Figure 14**:** The verifier as a detector of hallucinated final candidates: quality measures, confusion matrices, and the processing time spent on its errors.

D.5 Discrimination and calibration of the EWCS

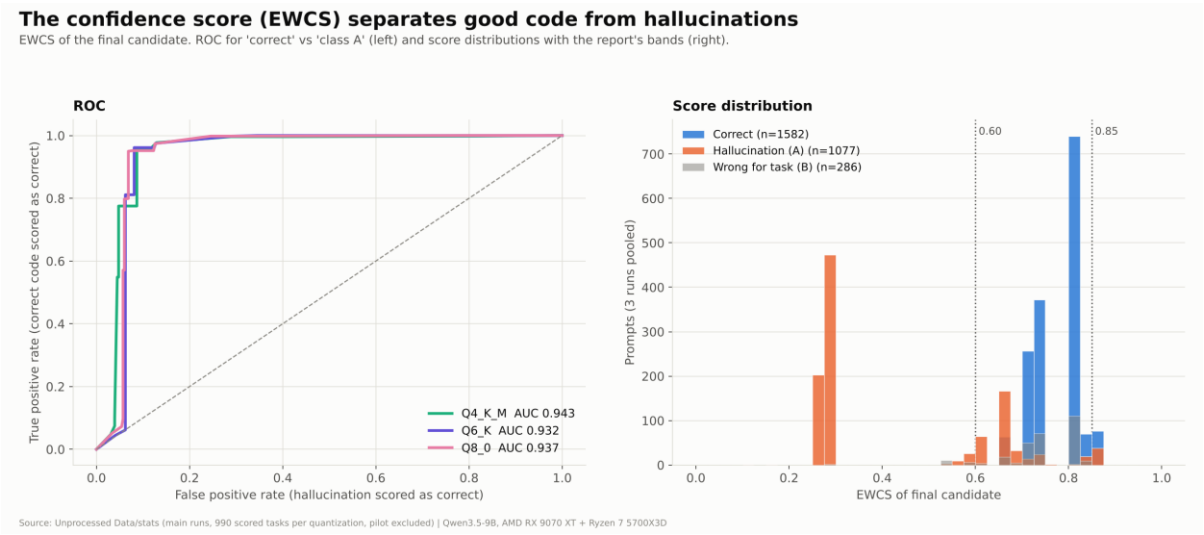


Figure 15**:** EWCS of the final candidate: ROC curve for correct versus class A, and score distributions with the decision thresholds (three runs pooled).

The EWCS separates correct from hallucinated final candidates with an area under the ROC curve of 0.932–0.943 (Figure 15). Hallucinated candidates cluster near 0.3, the value left when the access point is contradicted and the API layers that depend on it cannot be matched (Appendix C.6).

Table 21**:** Share of decided final candidates (correct, A or B) that were correct, by EWCS band (three runs pooled).

EWCS band	Candidates	Correct	Share correct
≥ 0.85 (deliver)	166	110	66.3%
0.60–0.85 (deliver with flag)	2,041	1,469	72.0%
< 0.60 (abstain)	738	3	0.4%

Candidates scoring 0.85 or more were correct only 66% of the time, *less* often than the flagged band, because the high band is reachable only for Petstore, the API with most residual errors and the most task-fidelity (class B) mistakes. The score measures how much of the claim the evidence covers, not how likely the code is to do what the user asked. The low band, by contrast, is almost pure error: 3 of 738 candidates below 0.60 were correct. Correct candidates cluster at the evidence ceilings of Table 19 (0.74 for curated, 0.825–0.838 for official specifications).

D.6 Sensitivity to the weights, thresholds and gate

The lecturer asked us to treat the six-layer score as a hypothesis, state the weights and thresholds in advance, report all planned schemes, and say whether any result depends on them. Figure 16 charts Table 6. The re-scoring is a fair counterfactual because the agent's only behavioural trigger, a correction, depends on contradictions and not on the score.

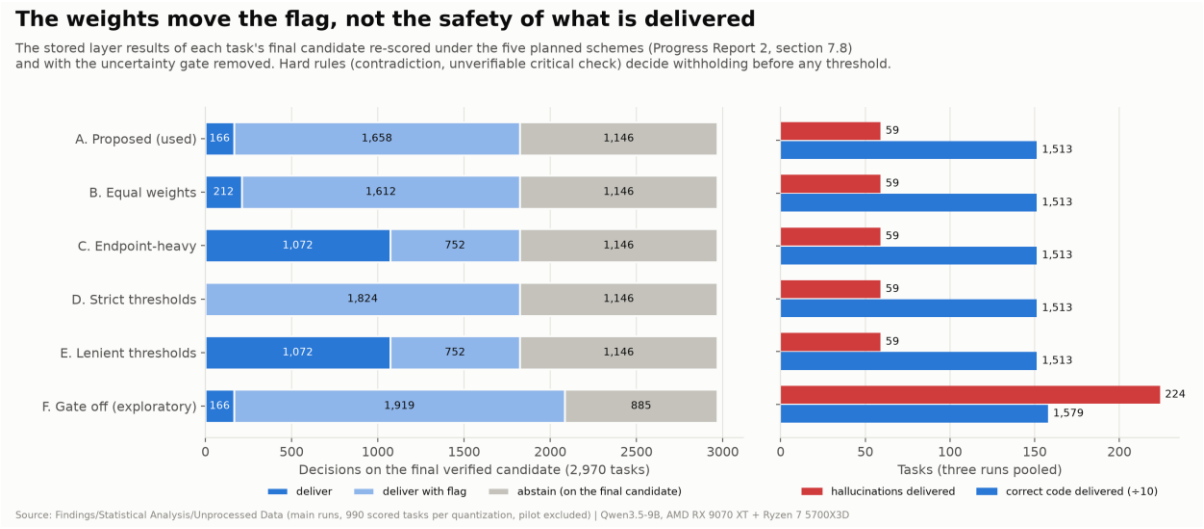


Figure 16: Decisions on the final candidate under the five planned schemes and with the uncertainty gate removed (three runs pooled).

D.7 Correction and agent behaviour

Table 22: Class of the first verified candidate → class of the final candidate, for tasks in which the code changed.

Transition	Q4_K_M	Q6_K	Q8_0
A → correct (hallucination repaired)	121	123	150
A → A (still hallucinated; withheld)	164	165	166
A → B (valid call, wrong task)	36	33	36
Runtime failure → correct	23	2	0
Corrections attempted / permitted	403 / 349	425 / 377	444 / 387
Final candidate correct after a correction	37.7%	34.1%	37.8%

Self-correction works about a third of the time. Figure 17 shows that more correction rounds rarely pay: prompts that needed two or three rounds cost 57–61 s and ended in correct code only 15% and 1% of the time. The model often bent the protocol (Figure 18): 34–39% of tasks reached the four-turn limit and needed the reply request, about 30% re-verified identical code, and 15–19% made more than one correction although the prompt permits one. Only 53–54% of tasks followed the protocol exactly.

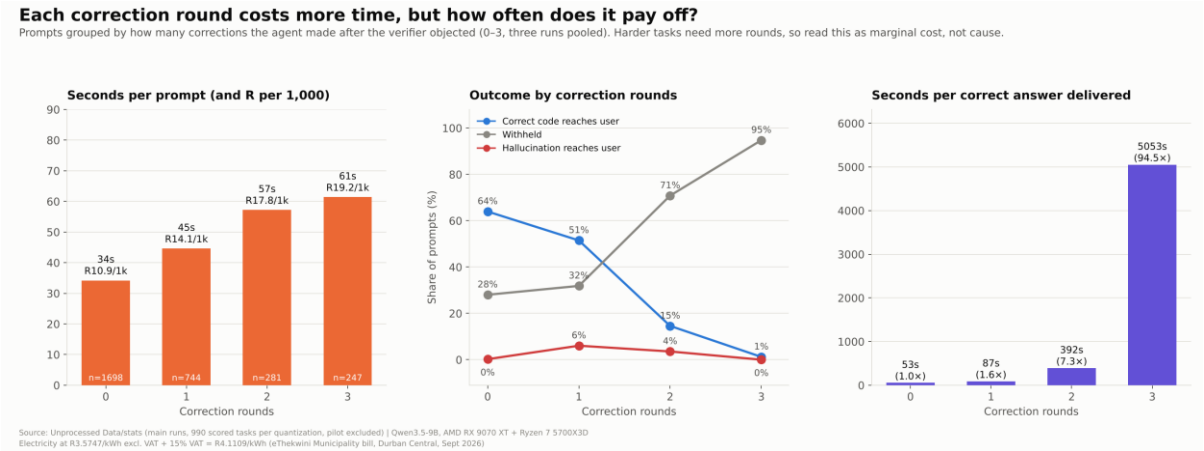


Figure 17: Cost and outcome by the number of correction rounds the agent made (three runs pooled).

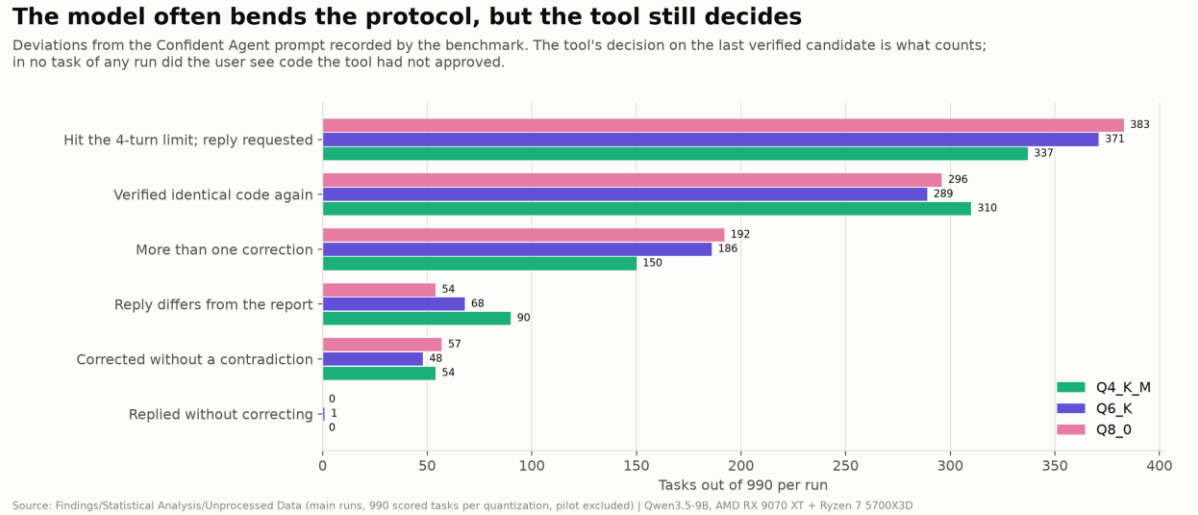


Figure 18**:** Deviations from the Confident Agent prompt recorded by the benchmark, per run.

D.8 Behaviour beyond API correctness

Table 23**:** Task-behaviour checks on code that reached the user (share of programs that make a request).

Requirement	Q4 base	Q4 agent	Q6 base	Q6 agent	Q8 base	Q8 agent
Returns the expected value	80%	83%	86%	82%	86%	84%
Returns None on an empty 204 response	12%	25%	21%	24%	22%	26%
Raises on a 500 error	100%	100%	100%	100%	100%	99%
All three met	8.7%	19.4%	18.5%	22.3%	17.7%	22.5%

Code reaching the user nearly always raises on a server error and usually returns the expected value, but few programs return `None` for an empty 204 response as instructed, in either condition. Among the agent's delivered programs that sent the right request, only 23–27% handled the empty response. Requiring the right request *and* all three behaviours, the agent delivered fully correct code for 10.3%, 10.5% and 11.3% of tasks (Q4_K_M, Q6_K, Q8_0), against 2.1%, 5.8% and 6.5% for the baseline (`stage_analysis.json`).

D.9 The cost of the intervention

Every prompt costs more under the intervention: in all 18 paired Wilcoxon tests (time, energy, total, generated, GPU and CPU energy, for each quantization) $p < 10^{-160}$, the rank-biserial correlation is at least 0.995, and 99.6–100% of individual prompts cost more (Figure 19).

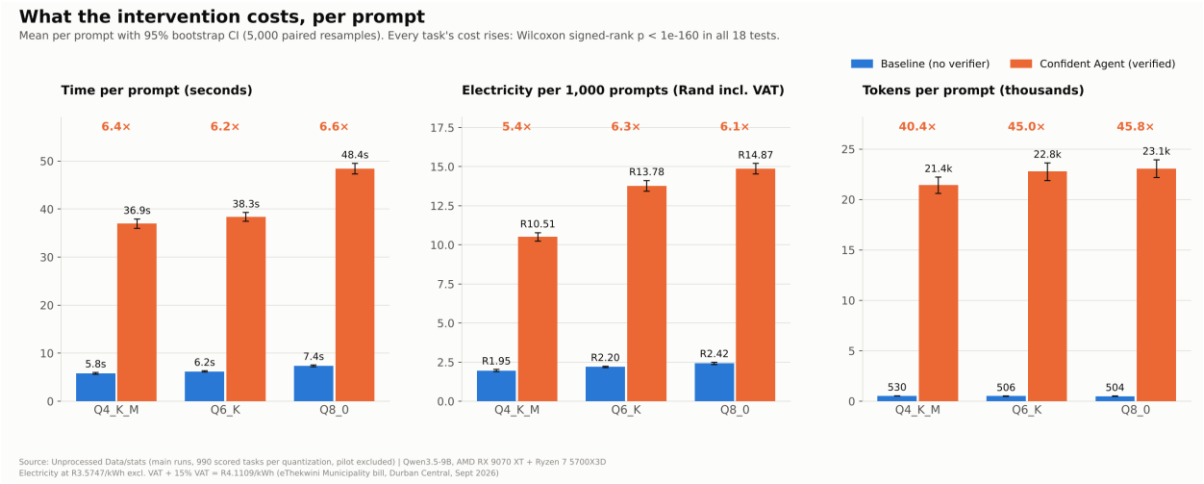


Figure 19**:** Time, electricity (Rand per 1,000 prompts) and tokens per prompt, with 95% bootstrap intervals.

Table 24**:** Cost of the intervention in time, energy and Rand (990 tasks per run; electricity at R4.1109/kWh incl. VAT).

Measure	Q4_K_M	Q6_K	Q8_0
Time per prompt, baseline → agent	5.8 → 36.9 s	6.2 → 38.3 s	7.4 → 48.4 s
Energy per prompt (GPU + CPU)	0.48 → 2.56 Wh	0.53 → 3.35 Wh	0.59 → 3.62 Wh
Electricity per prompt	0.20c → 1.05c	0.22c → 1.38c	0.24c → 1.49c
Tokens per prompt (input + output)	530 → 21,420	506 → 22,771	504 → 23,058
Time per correct answer delivered	23 → 73 s	17 → 76 s	21 → 94 s
Electricity per correct answer delivered	0.78c → 2.06c	0.60c → 2.73c	0.70c → 2.89c
Extra electricity per hallucination prevented	1.70c	2.39c	2.34c
Extra time per hallucination prevented	1.03 min	1.10 min	1.29 min
Whole run, 1,000 prompts: hours	1.6 → 10.2	1.7 → 10.7	2.0 → 13.5
Whole run, 1,000 prompts: Rand	R1.95 → R10.49	R2.19 → R13.79	R2.42 → R14.87

Measured energy covers the GPU and CPU packages only; wall-socket energy is higher. Whole-run figures include the 10 pilot tasks. Across the three runs the verifier added 29.0 hours and R32.59 of electricity (R28.34 before VAT).

At these rates, 10,000 prompts take 0.7–0.9 days on this machine without the verifier and 4.3–5.6 days with it (R20–R24 against R105–R149 of electricity). Figure 20 expresses the cost per good outcome.

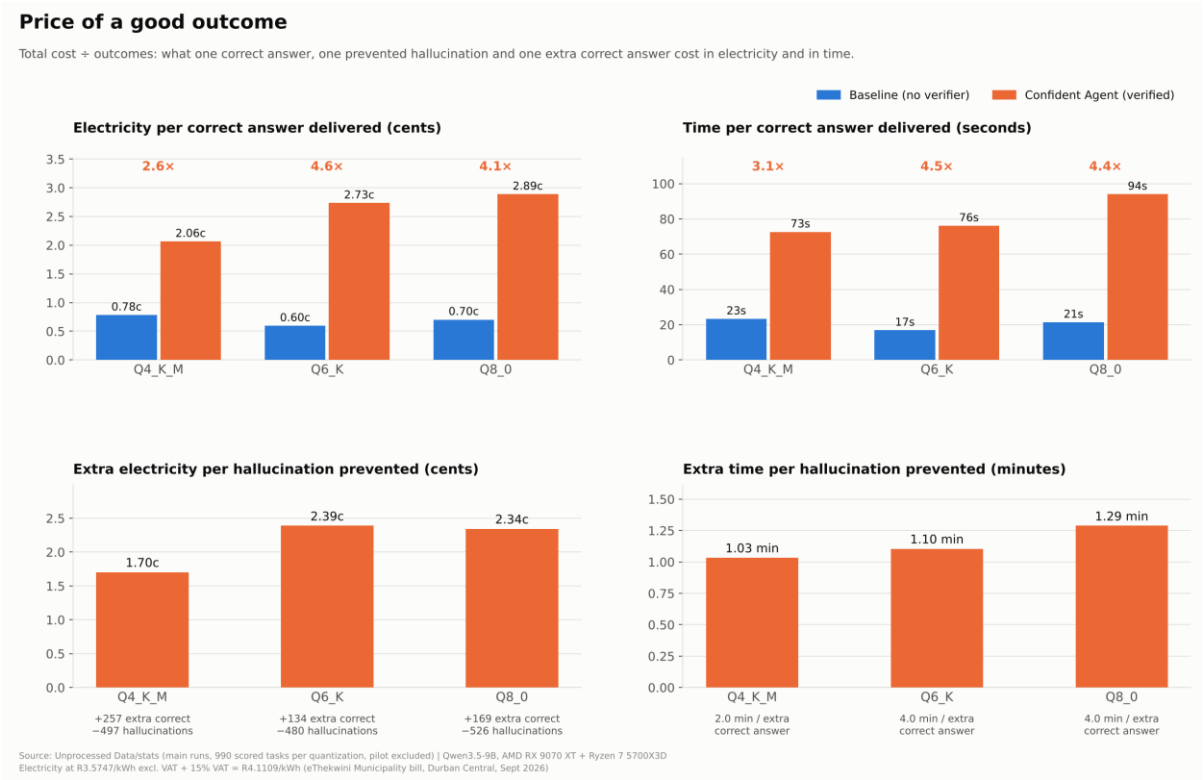


Figure 20**:** What one correct answer, one prevented hallucination and one extra correct answer cost in electricity and time.

Why the cost is so high. Verification takes 2.3–9.6 s per prompt (6–25% of the time). Each turn re-sends the whole growing context, including the six-layer report, so a verified prompt processes about 20,000–21,600 input tokens against 156 for the baseline (about 130 times as many) and generates 3.5–4.3 times as many tokens; input length alone tracks the time per prompt (Spearman ρ = 0.92). Energy rises less than tokens because input tokens are cheap to process. The 34–39% of tasks that hit the turn limit pay for the extra reply round as well.

Where the money goes. About 35–38% of the agent's processing time is spent on tasks that end in a correct delivery, and about half on tasks whose final candidate was a hallucination that was then withheld: 5.0–6.2 hours per run (R5.01–R6.65), money well spent from a safety standpoint but buying no deliverable (Figure 21). False alarms cost 0.4–1.0 hours per run. REST Countries alone consumed 7.1 hours and R7.93 across the three runs and delivered nothing.

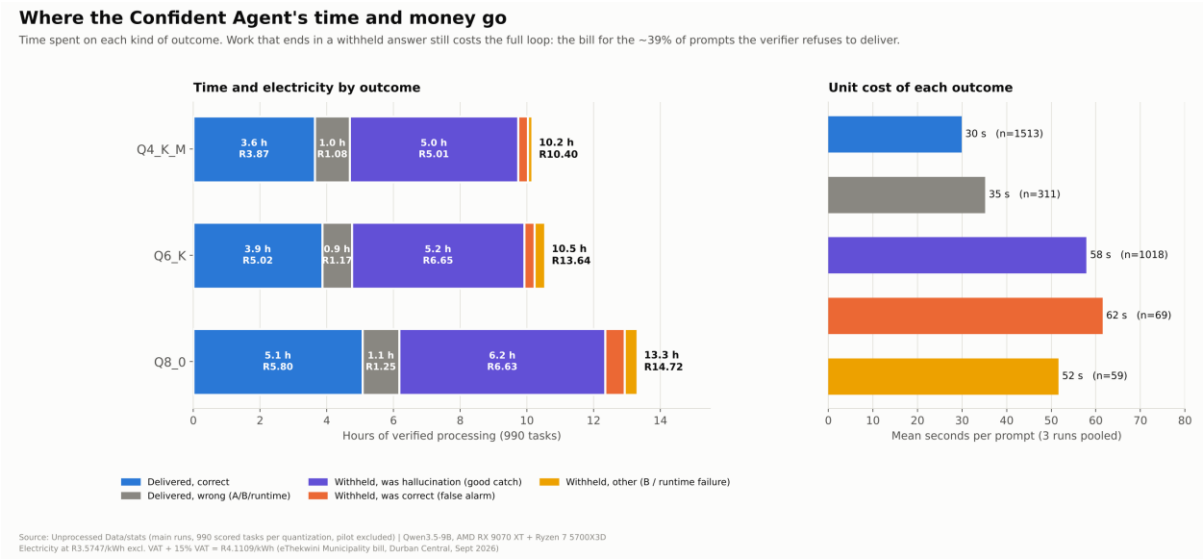


Figure 21**:** Where the Confident Agent's time and electricity go, by outcome.

D.10 Quantization, run-to-run reliability and agreement between runs

With the verifier, accuracy is almost independent of quantization (50.4–51.5% correct, 1.6–2.3% hallucinated), so the cheapest model is the best value: Q4_K_M needs 36.9 s and R10.51 per 1,000 prompts against 48.4 s and R14.87 for Q8_0, and delivers 48 correct answers per Rand of electricity against 35. Cost differences between quantizations are statistically clear (Kruskal–Wallis $p < 10^{-49}$ for time and energy in both conditions), but accuracy differences are descriptive: each quantization ran once, on a different day.

Run-to-run variation can be estimated because Q4_K_M and Q8_0 were each run twice with identical settings (the first series was superseded for its cost figures, Appendix B.9). Table 25 compares them. The headline rates agree within about two percentage points, but individual tasks vary: at temperature 0.2 without a fixed seed, 11–14% of tasks changed between delivered and withheld from one run to the next.

Table 25**:** Repeat runs with identical settings: first series (superseded) versus final series.

Measure	Q4_K_M	Q6_K	Q8_0
Hallucination reaching user, first series	50.8% → 1.6%	51.1% → 2.9%	53.1% → 2.0%
Hallucination reaching user, final series	51.8% → 1.6%	50.8% → 2.3%	55.2% → 2.0%
Correct code reaching user, first series	23.9% → 50.5%	37.1% → 48.8%	36.5% → 50.3%
Correct code reaching user, final series	24.9% → 50.9%	36.9% → 50.4%	34.4% → 51.5%
Same baseline class in both series (κ)	76.1% (0.63)	81.7% (0.70)	82.2% (0.70)
Same agent decision in both series (κ)	86.5% (0.75)	87.9% (0.78)	88.8% (0.79)
Tasks whose delivered/withheld status changed	134 (13.5%)	120 (12.1%)	111 (11.2%)

The first Q6_K run was slowed by an environmental fault but its accuracy is unaffected. Agreement statistics were computed before the manual review of the final series.

The three final runs are related in the same way. Table 26 compares each pair of quantization runs task by task: a task's outcome in one run strongly predicts its outcome in another, which is why the

runs are treated as a robustness check rather than as independent replications (Section 7.1). Of the 990 tasks, the baseline hallucinated on 412 in all three runs, on 363 in none, and on 215 in one or two.

Table 26**:** Agreement between the three final runs, task by task (990 tasks; Cohen's κ in brackets).

Per-task outcome	Q4_K_M vs Q6_K	Q4_K_M vs Q8_0	Q6_K vs Q8_0
Baseline hallucinated (yes / no)	84.4% (0.69)	84.9% (0.70)	87.2% (0.74)
Baseline class	69.3% (0.51)	69.6% (0.51)	81.4% (0.68)
Agent delivered or withheld	82.1% (0.62)	83.1% (0.64)	86.3% (0.71)

Computed by `Findings/EvaluateBenchmark/stage_analysis.py` from the per-task evaluation files.

D.11 Secondary outcomes and effect sizes

Table 27**:** Secondary outcomes (exact McNemar tests; intervals over tasks).

Outcome		Q4_K_M	Q6_K	Q8_0
Class A or B reaching the user	rate	59.8% → 11.1%	56.5% → 9.4%	61.0% → 9.5%
	p	1.5×10^{-107}	6.1×10^{-112}	4.7×10^{-128}
Any error (A, B or runtime failure)	rate	74.9% → 11.5%	62.8% → 10.1%	65.4% → 9.8%
	p	1.9×10^{-164}	5.4×10^{-134}	2.9×10^{-140}
Correct code reaching the user (H2)	rate	24.9% → 50.9%	36.9% → 50.4%	34.4% → 51.5%
	gain [CI]	+26.0 [22.7, 29.2]	+13.5 [10.7, 16.5]	+17.1 [13.9, 20.2]
	p	3.2×10^{-51}	6.4×10^{-20}	5.7×10^{-26}
Class B alone reaching the user	count	79 → 94	56 → 70	58 → 74

Hallucinations and runtime failures fall sharply, so the combined error rates fall, but class B errors alone rise slightly: a verifier that checks the API contract does not check the task (Section 8.3). Figure 22 shows the paired differences with both kinds of interval.

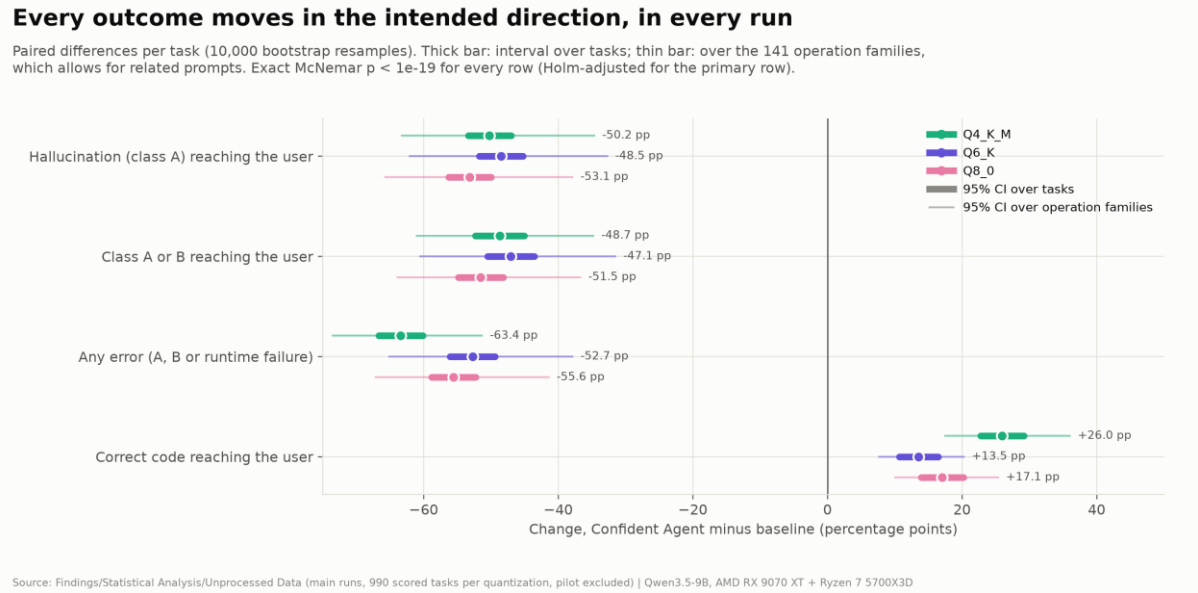


Figure 22**:** Paired differences for the primary and secondary outcomes, with 95% bootstrap intervals over tasks (thick) and operation families (thin).

D.12 Cost statistics and reviewer agreement

H4 is quantified rather than tested for direction. Paired Wilcoxon signed-rank tests on time, energy and tokens give $p < 10^{-160}$ in every one of 18 comparisons with rank-biserial correlations of at least 0.995; the ratio-of-means intervals are narrow (for example, time ×6.36 [6.13, 6.58] at Q4_K_M). Quantization affects cost significantly in both conditions (Kruskal–Wallis, Holm-adjusted pairwise tests significant for time and energy), but, with one run per quantization, those differences cannot be separated from day-to-day machine conditions.

Agreement between the AI-assisted first review and the independent second review on the 46 decisive candidates was 43/46, Cohen's κ = 0.88, which is "almost perfect" agreement on the Landis and Koch (1977) scale. Agreement between repeated runs of the same model and settings was κ = 0.63–0.70 for the baseline class and 0.75–0.79 for the agent's decision ("substantial").

Appendix E: Changes since Progress Report Two

Progress Report Two was written before implementation. Figure 23 summarises the path from the frozen dataset to the final analysis, and Table 28 lists every point on which this report supersedes Progress Report Two, with the reason.

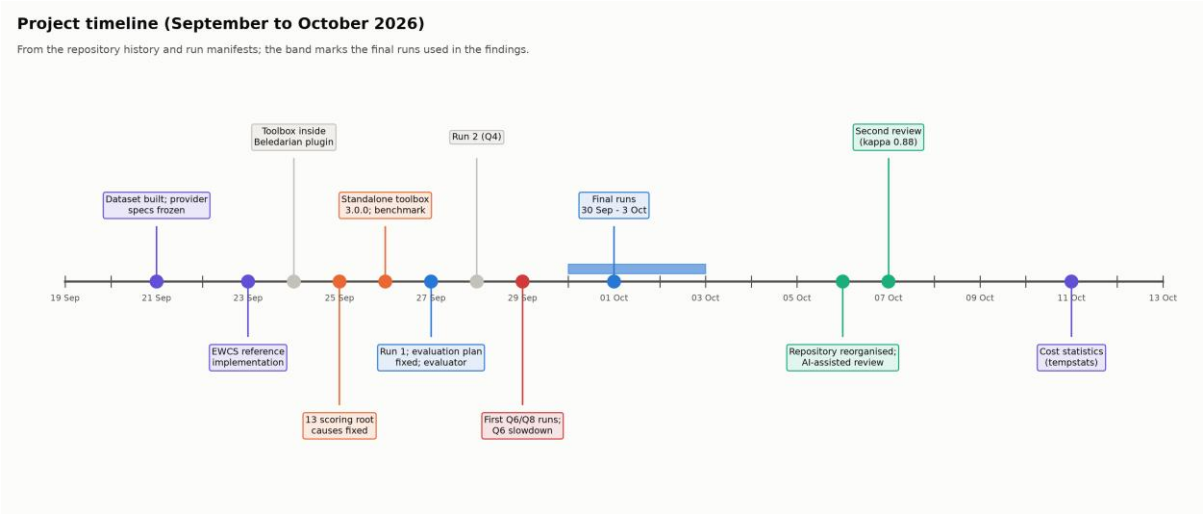


Figure 23**:** Project timeline from the frozen dataset to the final analysis. The shaded band marks the three final benchmark runs on which the findings rest.

Table 28**:** Design decisions in Progress Report Two and in the final system.

Topic	Progress Report Two	Final	Why
Model	Qwen3.5-9B-Instruct; temperature 0.3; top-p 0.9; context 8,192; fixed seed	Qwen3.5-9B GGUF at Q4_K_M, Q6_K, Q8_0; temperature 0.2; context 30,208; no seed; thinking off	Tool conversations need a longer context; one pinned configuration for every run (lecturer item 2)
Baseline prompt	"You are a code generation assistant. Provide only code."	Two-line API coding-assistant prompt asking for one python block	Reliable code-block extraction
Evidence	Specifications "available"; later dynamic discovery considered	Frozen, hashed snapshots bundled with the toolbox and selected by host	Reproducibility; four APIs publish no discoverable specification (lecturer item 4)
Static checker	Pylance via the Language Server Protocol	Pyright 1.1.414 command line (Pylance's engine)	Scriptable and pinnable
Tool attachment	MCP tool	LM Studio tools provider; function calling through a bridge in the experiment	Same interface without an extra server (Appendix C.2)
Layer 2 tier	typeshed via Pylance, tier 2 (0.8) in the worked example	Absence of a Pyright error, tier 4 (0.5)	A clean type check is weak evidence
Missing or failed layer	Excluded from the score (Python reference)	Counted as unverifiable; the decision abstains	A layer that did not run must not look like a clean result
Decision policy	Safety constraint, then thresholds 0.85/0.60	Safety constraint, **uncertainty gate**, then thresholds	Without the gate, unverifiable candidates were delivered (Section 8.4)
Correction	At most one, by the controller	At most one, by the model under the prompt; deviations measured	Deployable package; tool keeps the delivery decision

Topic	Progress Report Two	Final	Why
Primary metric	Unsafe Delivery Rate	Class A code reaching the user, per task, by an independent evaluator	Same idea, defined with explicit classes A, B, C
Evaluation	Static comparison and H01–H10 codes	Sandboxed execution with a recording stand-in; H-codes mapped to error kinds	Exact requests, including conditional code (lecturer item 5)
Dataset targets	40/40/20% difficulty; 55% no authentication; 10% manual review	Actual composition reported; 100-task review not completed	Do not relabel tasks to hit targets
Statistics	McNemar (pending research)	Exact McNemar, task and family bootstrap, Holm, Wilson, Wilcoxon, AUC, κ	Paired binary outcome; related prompts
Sensitivity analysis	Planned (five schemes)	Done, plus the gate (Section 8.4)	Lecturer item 6
Multi-agent reframing	Chief agent with specialist agents	One model, one deterministic verification tool	Deterministic checks are stronger evidence than another model

Appendix F: Example Task and Prompts

Task `REST_PY_GH_0003` (operation family `06ffd07d…`, difficulty heuristic "medium", provider specification `github.openapi.json`, SHA-256 `4fbdc7d0…`). The model sees only the instruction:

```
Write a Python function solve(inputs) for GitHub REST API version 2026-03-10. Delete an
organization. The inputs dictionary provides: organization: The organization name. Use the
caller-supplied credential in inputs["credential"], assumed to have the necessary
permissions.
Return the decoded JSON response. Return None for a successful empty response. Raise an
exception
for an unsuccessful HTTP response. Assume referenced resources and accounts exist. Do not
execute
the function.
```

The answer key holds the expected request for the test inputs `{"organization": "example-org", "credential": "EXAMPLE_CREDENTIAL_NOT_REAL"}`:

```
DELETE https://api.github.com/orgs/example-org
Accept: application/vnd.github+json
X-GitHub-Api-Version: 2026-03-10
Authorization: Bearer EXAMPLE_CREDENTIAL_NOT_REAL
```

and a reference solution, generated from the specification by template:

```
import requests
from urllib.parse import quote

def solve(inputs):
    url = 'https://api.github.com/orgs/{org}'
    headers = {}
    headers['Accept'] = 'application/vnd.github+json'
    headers['X-GitHub-Api-Version'] = '2026-03-10'
    headers['Authorization'] = 'Bearer ' + inputs["credential"]
    url = url.replace('{org}', quote(str(inputs['organization']), safe=''))
    response = requests.request('DELETE', url, params={}, headers=headers, timeout=20)
    response.raise_for_status()
    if response.status_code == 204 or not response.content:
        return None
    return response.json()
```

The Confident Agent system prompt (194 lines, in `Code Submission/confident-agent-toolbox/instructions`) prescribes this workflow: write the complete candidate with `requests`; do not look for or write a specification (evidence is selected by host from the eight listed hosts); call `verify_candidate` exactly once with the code only; if a layer is `verified_incorrect` on the first candidate, fix exactly those problems and verify once more; otherwise, or after the second result, reply with the tool's `report` verbatim. It states that the tool does not check task behaviour and that the EWCS is not a probability.

Appendix G: Run Artefacts and Provenance

Table 29**:** Identifiers of the frozen inputs (SHA-256 prefixes; full hashes in the run manifests and the dataset).

Artefact	SHA-256	Notes
Task file (answer key) rest_api_tasks_v1_DRAFT.jsonl	7a09f2bb…	LF line endings
Prompt file rest_api_prompts_v1_DRAFT.jsonl	c4c68a64…	identical in both conditions
Confident Agent system prompt (as sent)	0ebc4dce…	operator note removed
Frozen PyPI evidence (requests)	ef06670a…	interface layer
github.openapi.json	4fbdc7d0…	13.0 MB, 819 paths
petstore.openapi.json	00441c05…	OpenAPI 3.0.4
openmeteo-forecast / air-quality / marine	f57c2918… / ee203da7… / 096a28c7…	converted from YAML
jsonplaceholder / dummyjson / restcountries (curated)	d2dfbc53… / d313813a… / d02f4ab3…	researcher-authored subsets

Run folders (`Findings/Benchmark/Completed Runs/<model>/`) hold one zip per condition with every reply, tool call, verified candidate and verification result, and a run manifest recording the settings, LM Studio's model information, the machine and the power sensors. The final runs were started from repository commits `d954dc0` (Q6_K), `f6ee6b7` (Q8_0) and `7e928f2` (Q4_K_M); the toolbox and benchmark code are identical in all three (benchmark 2026-09-27.2, toolbox 3.0.0).

Appendix H: Error Kinds Used by the Evaluator

Table 30**:** Kinds of error recorded in the code reaching the user, read from the evaluator's findings with fixed rules.

Class	Kinds
A	wrong host, version or base path; invented endpoint or method; invented parameter or body field; invalid value or structure; missing required part or credential; undocumented response field; non-existent `requests` API
B	different declared endpoint; method not acceptable for the task; omitted parameter the task specifies; different parameter value; omitted request body; unrequested request body; request body differs; extra requests; credential missing or malformed where the API's requirement is unknown; JSON body sent as form data; insecure scheme (http)
Runtime failure	input used as the wrong type; reads an input the task does not provide; syntax error; makes no request; does not finish (timeout); blocked operation; undefined name; raises its own error before the request; wrong call or argument types

Appendix I: Manual Review Rules

Each question that a curated specification left open was answered once and applied to every candidate asking it (`Findings/EvaluateBenchmark/reviews/review_rules.py`). The rules, with their evidence:

- **DummyJSON query parameters.** The documentation lists `limit`, `skip`, `select`, `sortBy`, `order` and `q`. Names such as `sort`, `sort_by`, `page`, `fields`, `include` and `_fields` are class A; a live check showed they have no effect (`?sort=title` returns the unsorted list, `?sortBy=title` sorts it).
- **DummyJSON and JSONPlaceholder body fields.** A field the resource has (for example `gender` on a user, `id` on a todo) is an extra documented body field, class B; a field it does not have (for example `title` on a DummyJSON todo, `userId` on a JSONPlaceholder photo) is class A.
- **Response fields.** DummyJSON returns the resource object itself; reading `users`, `post`, `success`, `statusCode` or `content` from a single-resource response is class A.
- **JSONPlaceholder filters and routes.** Filtering by any resource field is documented ("basic filtering is supported through query parameters") and works live; filtering by a non-field (`ids`) is ignored and class A. Nested routes such as `/posts/1/comments` are documented and real; using one where the task expected another endpoint is class B.
- **REST Countries paths.** Every v5 endpoint lies under `/countries/v5` (list, search, aggregate, read and search by property); paths such as `/v5/countries`, `/v5.0/search` or `/v3.1/all` on `api.restcountries.com` do not exist and are class A.

The first review classified the 46 candidates that reached the user as 24 class A and 22 class B; the second reviewer agreed on 43, and the group settled the three JSONPlaceholder list-endpoint cases as correct. Of the 350 withheld candidates, 327 were class A and 23 class B.

Appendix J: Reproducing the Results

The submission folder `COMP301 Project Final Submission` contains `Code Submission` (toolbox, dataset builder), `Findings` (benchmark with completed runs, evaluator, statistical analysis) and `Final Report and Documentation`. The evaluator and statistics need only Python 3.9 or later; the charts need pandas, NumPy, SciPy, Matplotlib and seaborn.

```
cd "COMP301 Project Final Submission\Findings\EvaluateBenchmark"
python evaluate_benchmark.py              # sandboxed scoring of every run (about 12
minutes)
python compile_stats.py                   # CSV tables in Statistical Analysis\Unprocessed
Data
python stage_analysis.py                  # first candidates, agreement between runs
(Section 8.2)
cd "..\Statistical Analysis"
python -X utf8 tempstats\build_tempstats.py                # cost charts and tables
python -X utf8 tempstats\report_figures\build_report_figures.py   # charts R01-R05
cd "..\..\Final Report and Documentation\Final Report\build"
python -X utf8 build_diagrams.py          # diagrams D01-D08
node build_report.js report.docx          # this report (then update fields in Word)
```

Rerunning the benchmark itself requires LM Studio with the Qwen3.5-9B quantizations, the toolbox's Python environment and LibreHardwareMonitor for energy; the procedure is in `Findings/Benchmark/README.md`.

Appendix K: Glossary

Table 31**:** Glossary.

Term	Meaning
Abstention	The agent withholds code because evidence contradicts it or cannot establish it.
Baseline	The same model with a minimal system prompt and no verification.
Class A / B / C	API hallucination / valid call that is wrong for the task / equivalent to the expected request (Table 1).
Confident Agent	The model plus the Confident Agent prompt and the `verify_candidate` tool.
Decision policy	The ordered rules that turn verification results into deliver, deliver with flag, correct or abstain: safety constraint, uncertainty gate, then thresholds.
EWCS	Evidence-Weighted Confidence Score: the share of applicable verification weight supported by positive evidence. Not a probability.
First candidate	The first code the model submitted for verification in a task, written under the Confident Agent prompt before any verification feedback.
Frozen evidence	Specification files stored with their hash and never changed during the experiment.
Incorrect acceptance / rejection	A hallucinated candidate delivered / a correct candidate withheld.
McNemar test	Test for a change in a paired binary outcome, using only the discordant pairs.
Operation family	A group of tasks that exercise the same underlying API operation.
Safety constraint	The policy rule that never releases code with a layer verified incorrect.
Uncertainty gate	The policy rule that withholds code when any applicable verification layer is unverifiable.
Verified correct / incorrect / unverifiable / not applicable	The four states of a verification layer.